Data preparation:

This notebook makes plots/analysis of some initial data preparation.

This first section (below) covers the filtering and foldseek outputs. I filtered out amino acids with over 300 AA and also removed ligands, HIS-tags, and water molecules.

Next, I include the initial analysis of the RMSD distributions of a generated synthetic dataset of backbone variations (created with rosetta backrub). This notebook jsut covers the output of a subset of 30 proteins that were used to created 150 conformations. I have a larger compute job to generate 5 conformations per protein in a datset of 300k protein.

Finally, I include some reprsentations from existing "expert" models, protpardelle, which will be used to assess representation alignment. The idea here is to compare how the embeddings will look across different models (recent papers showcase that reprsentations for similar models (eg all the LLMs) are similar, which could be contributed by the fact that these models do have the same baseline transformer architecture and are trained on the same datasets for the most part...In my case, I want to see if protein-related models will also have similar embeddings, and particualrly, how these embeddings will look for "edge" cases such as switch proteins.)

CATH20 Dataset - Foldseek Structural Analysis

Quick visualization of the filtered CATH20 dataset (cath20-filtered-foldseek) (filter_dataset.py and foldseek_tm_filter.py) Loads pre-computed foldseek all-vs-all results and generates distribution plots, pairwise heatmaps, correlation analysis, and structural embeddings.

In [45]:
```
%matplotlib inline

from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

```
from sklearn.manifold import MDS
from umap import UMAP
```

In [46]:
```
FOLDSEEK_COLUMNS = [
    "query", "target", "fident", "alnlen", "mismatch", "gapopen",
    "qstart", "qend", "tstart", "tend", "evalue", "bits",
    "alntmscore", "rmsd", "qaln", "taln", "qlen", "tlen",
]


def load_foldseek_results(results_tsv):
    """Load foldseek results TSV into a DataFrame."""
    df = pd.read_csv(results_tsv, sep="\t", header=None, names=FOLDSEE
    df["query"] = df["query"].astype(str).str.replace(r"\.pdb$", "", r
    df["target"] = df["target"].astype(str).str.replace(r"\.pdb$", "",
    return df


def build_pairwise_matrix(df, metric, proteins):
    """Build a symmetric pairwise matrix from alignment results."""
    n = len(proteins)
    prot_idx = {p: i for i, p in enumerate(proteins)}

    if metric == "rmsd":
        mat = np.full((n, n), np.nan)
        np.fill_diagonal(mat, 0.0)
        best = df.groupby(["query", "target"])[metric].min()
    else:
        mat = np.zeros((n, n))
        np.fill_diagonal(mat, 1.0)
        best = df.groupby(["query", "target"])[metric].max()

    for (q, t), val in best.items():
        if q in prot_idx and t in prot_idx:
            i, j = prot_idx[q], prot_idx[t]
            mat[i, j] = val
            mat[j, i] = val

    return mat


def plot_metric_distribution(df, metric, label, units, color="#2a9d8f"
    """Plot histogram of a structural metric (excluding self-hits)."""
    non_self = df[df["query"] != df["target"]].dropna(subset=[metric])
    vals = non_self[metric]

    fig, ax = plt.subplots(figsize=(9, 4))
    sns.histplot(vals, bins=80, color=color, edgecolor="white", linewi
    ax.set_xlabel(f"{label} ({units})")
    ax.set_ylabel("Count")
    ax.set_title(f"{label} distribution (n={len(vals)} non-self alignm
    fig.tight_layout()
```

```
    plt.show()


def plot_pairwise_heatmap(matrix, proteins, metric_label, cmap="viridi
    """Plot a pairwise heatmap from a square matrix."""
    n = len(proteins)
    show_labels = n <= 50

    fig, ax = plt.subplots(figsize=(10, 9))
    im = ax.imshow(matrix, cmap=cmap, aspect="auto", vmin=vmin, vmax=v
    fig.colorbar(im, ax=ax, label=metric_label)

    if show_labels:
        ax.set_xticks(range(n))
        ax.set_yticks(range(n))
        ax.set_xticklabels(proteins, rotation=90, fontsize=6)
        ax.set_yticklabels(proteins, fontsize=6)
    else:
        ax.set_xlabel("Protein index")
        ax.set_ylabel("Protein index")

    ax.set_title(f"Pairwise {metric_label} (n={n})")
    fig.tight_layout()
    plt.show()


def plot_correlation_heatmap(df, title="Structural Metric Correlations
    """Compute and plot correlation heatmap of structural metrics."""
    non_self = df[df["query"] != df["target"]]
    metrics = {
        "TM-score": "alntmscore",
        "RMSD": "rmsd",
        "Seq Identity": "fident",
        "Alignment Length": "alnlen",
        "E-value": "evalue",
        "Bit Score": "bits",
    }
    metric_df = non_self[list(metrics.values())].rename(columns={v: k
    corr = metric_df.corr(method="pearson")

    fig, ax = plt.subplots(figsize=(8, 7))
    sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r",
                vmin=-1, vmax=1, center=0, square=True, ax=ax, linewid
    ax.set_title(title)
    fig.tight_layout()
    plt.show()
    display(corr.round(3))
    return corr


def plot_embedding(dist_matrix, proteins, method="UMAP"):
    """Plot a 2D embedding from a distance matrix."""
```

```
    mat = dist_matrix.copy()
    mat[np.isnan(mat)] = np.nanmax(mat)
    n = len(proteins)

    if method == "UMAP":
        n_neighbors = min(15, n - 1)
        reducer = UMAP(n_components=2, metric="precomputed", n_neighbo
    else:
        reducer = MDS(n_components=2, dissimilarity="precomputed", ran

    coords = reducer.fit_transform(mat)

    fig, ax = plt.subplots(figsize=(10, 8))
    ax.scatter(coords[:, 0], coords[:, 1], s=15, alpha=0.7, c="steelbl
    ax.set_xlabel(f"{method} 1")
    ax.set_ylabel(f"{method} 2")
    ax.set_title(f"{method} embedding of structural similarity (n={n})

    if n <= 30:
        for i, prot in enumerate(proteins):
            ax.annotate(prot, (coords[i, 0], coords[i, 1]), fontsize=5

    fig.tight_layout()
    plt.show()
    return coords


def plot_residue_alignment_per_protein(df, max_proteins=50):
    """Plot per-protein residue alignment coverage as a bar chart."""
    non_self = df[df["query"] != df["target"]].copy()
    non_self["aln_frac"] = non_self["alnlen"] / non_self["qlen"]
    mean_cov = non_self.groupby("query")["aln_frac"].mean().sort_value
    mean_cov = mean_cov.head(max_proteins)

    fig, ax = plt.subplots(figsize=(12, max(5, len(mean_cov) * 0.25)))
    ax.barh(range(len(mean_cov)), mean_cov.values, color="steelblue")
    ax.set_yticks(range(len(mean_cov)))
    ax.set_yticklabels(mean_cov.index, fontsize=7)
    ax.set_xlabel("Mean fraction of residues aligned")
    ax.set_title("Per-Protein Residue Alignment Coverage")
    ax.invert_yaxis()
    fig.tight_layout()
    plt.show()
```

Load CATH20 foldseek results

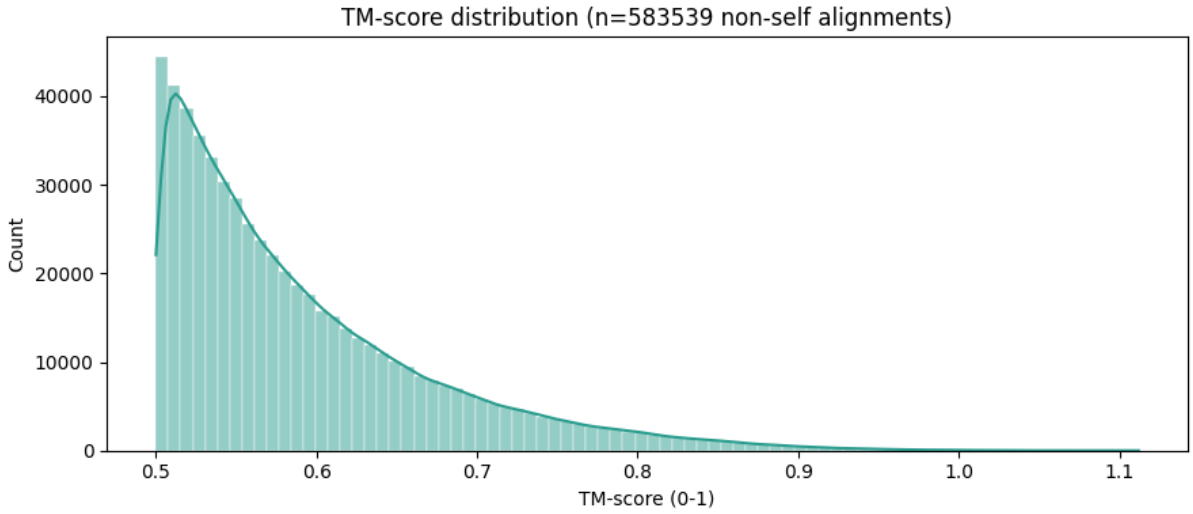
In [47]:
```
PDB_DIR = Path("/Users/joycemo/Documents/PhD/Rotation3/dataset/cath20/
RESULTS_TSV = PDB_DIR / "foldseek_results.tsv"

df = load_foldseek_results(RESULTS_TSV)
all_proteins = sorted(df["query"].unique())
```

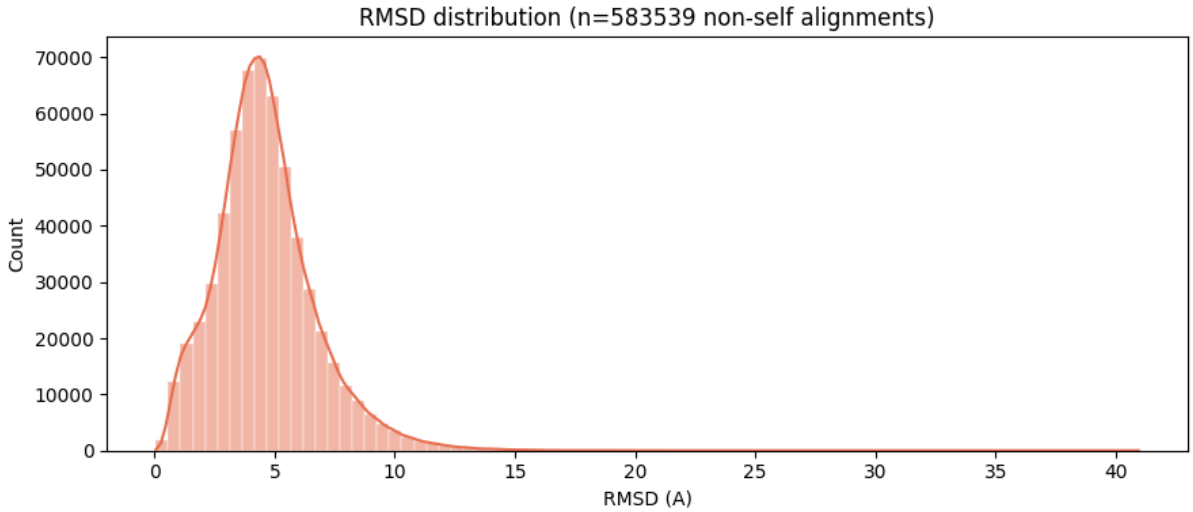
```
non_self = df[df["query"] != df["target"]]

print(f"Loaded {len(df)} alignments across {len(all_proteins)} unique
print(f"Non-self alignments: {len(non_self)}")
print()

for col, label in [("alntmscore", "TM-score"), ("rmsd", "RMSD"), ("fid
    vals = non_self[col].dropna()
    print(f"{label}: mean={vals.mean():.3f}, median={vals.median():.3f
          f"min={vals.min():.3f}, max={vals.max():.3f}")
```

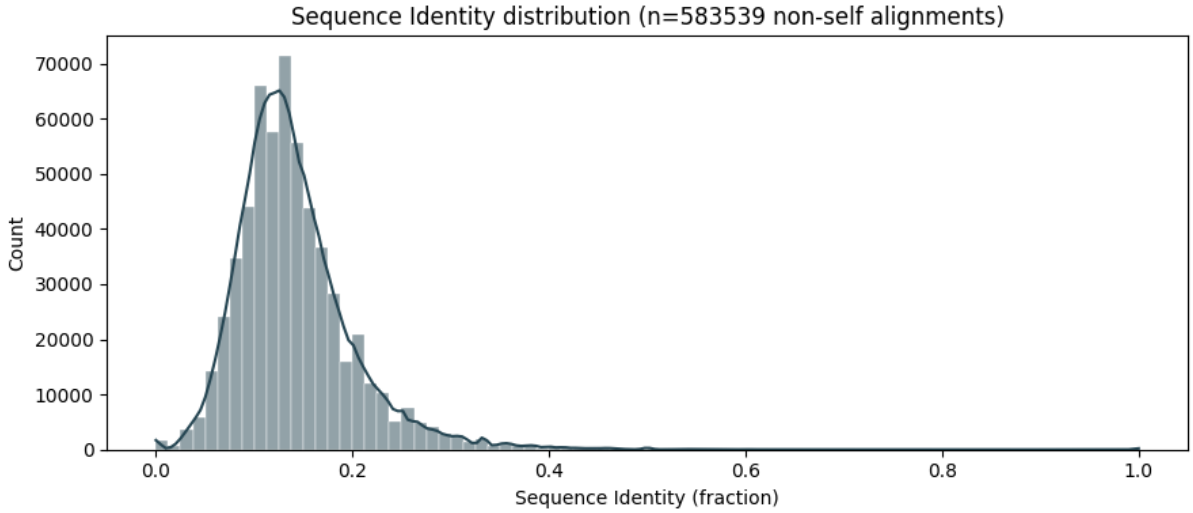
```
Loaded 597057 alignments across 13518 unique proteins
Non-self alignments: 583539

TM-score: mean=0.591, median=0.566, min=0.500, max=1.112
RMSD: mean=4.642, median=4.437, min=0.063, max=40.950
Seq identity: mean=0.141, median=0.131, min=0.000, max=1.000
```

Distributions

In [48]:
```
plot_metric_distribution(df, "alntmscore", "TM-score", "0-1", color="#
plot_metric_distribution(df, "rmsd", "RMSD", "A", color="#e76f51")
plot_metric_distribution(df, "fident", "Sequence Identity", "fraction"
```

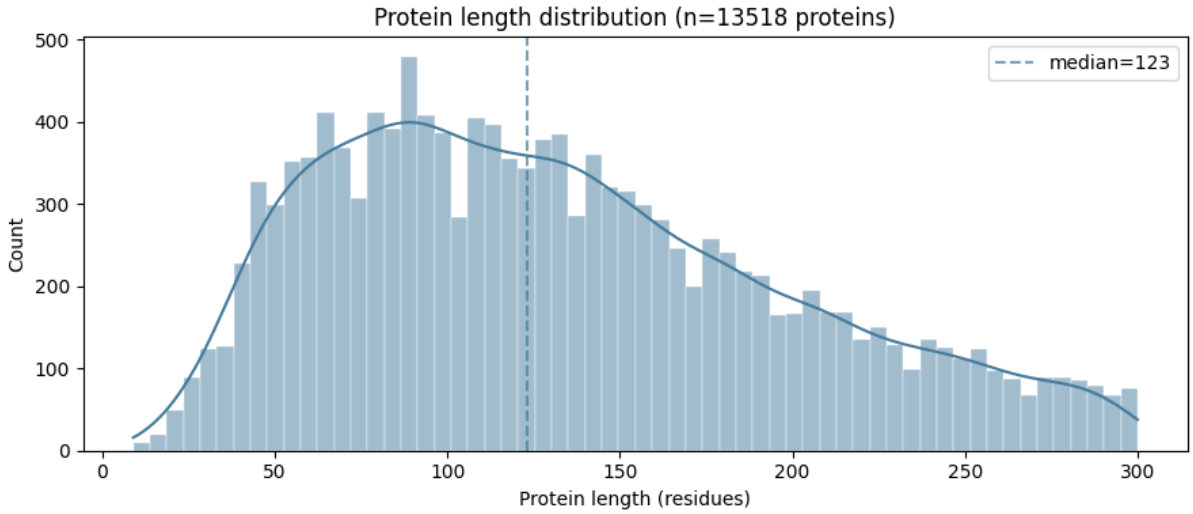






Protein length distribution

In [49]:
```
lengths = df.groupby("query")["qlen"].first()

fig, ax = plt.subplots(figsize=(9, 4))
sns.histplot(lengths, bins=60, color="#457b9d", edgecolor="white", lin
ax.set_xlabel("Protein length (residues)")
ax.set_ylabel("Count")
ax.set_title(f"Protein length distribution (n={len(lengths)} proteins)
ax.axvline(lengths.median(), color="#457b9d", linestyle="--", alpha=0.
ax.legend()
fig.tight_layout()
plt.show()

print(f"Length: mean={lengths.mean():.0f}, median={lengths.median():.0
      f"min={lengths.min()}, max={lengths.max()}")
```

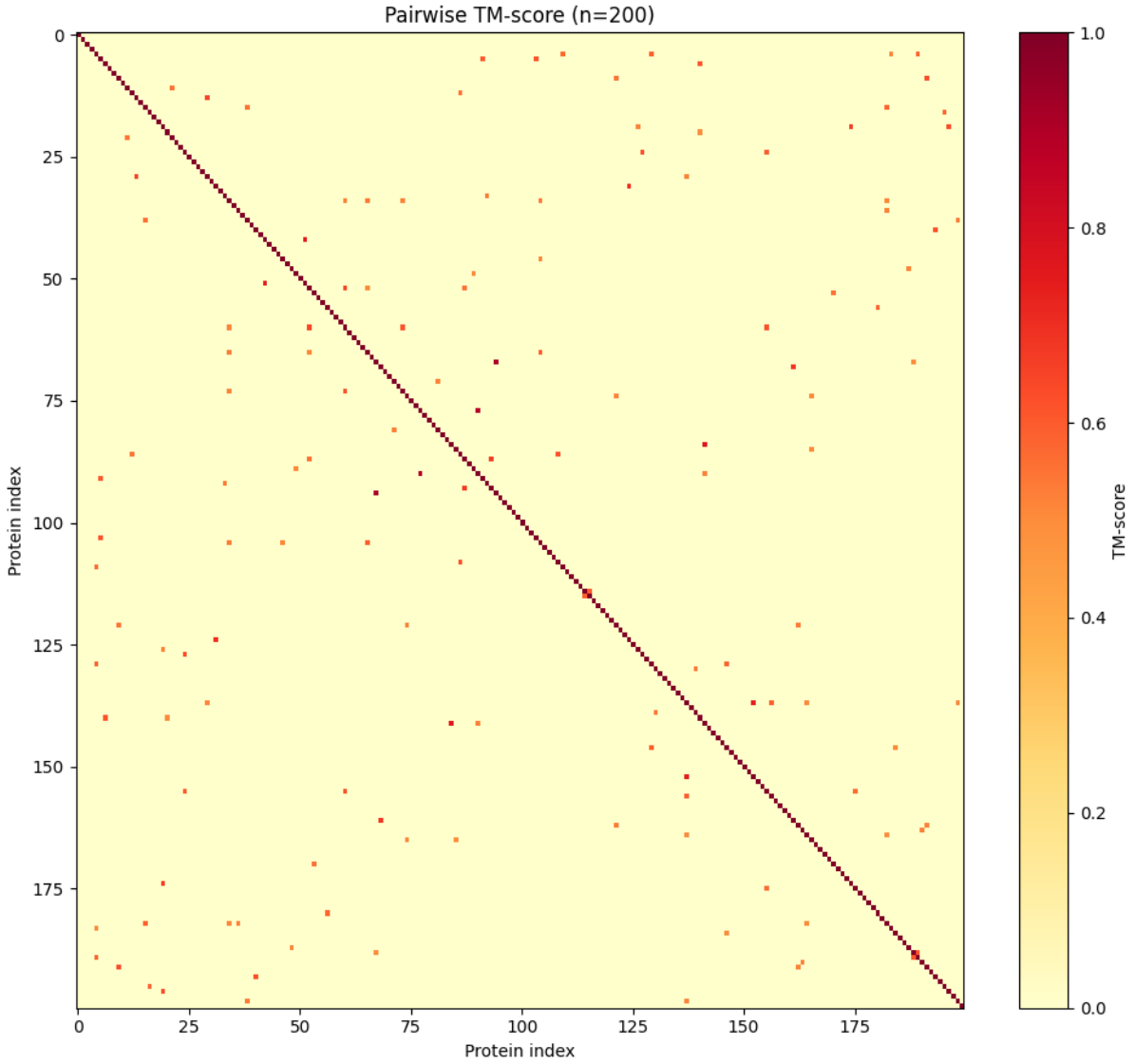


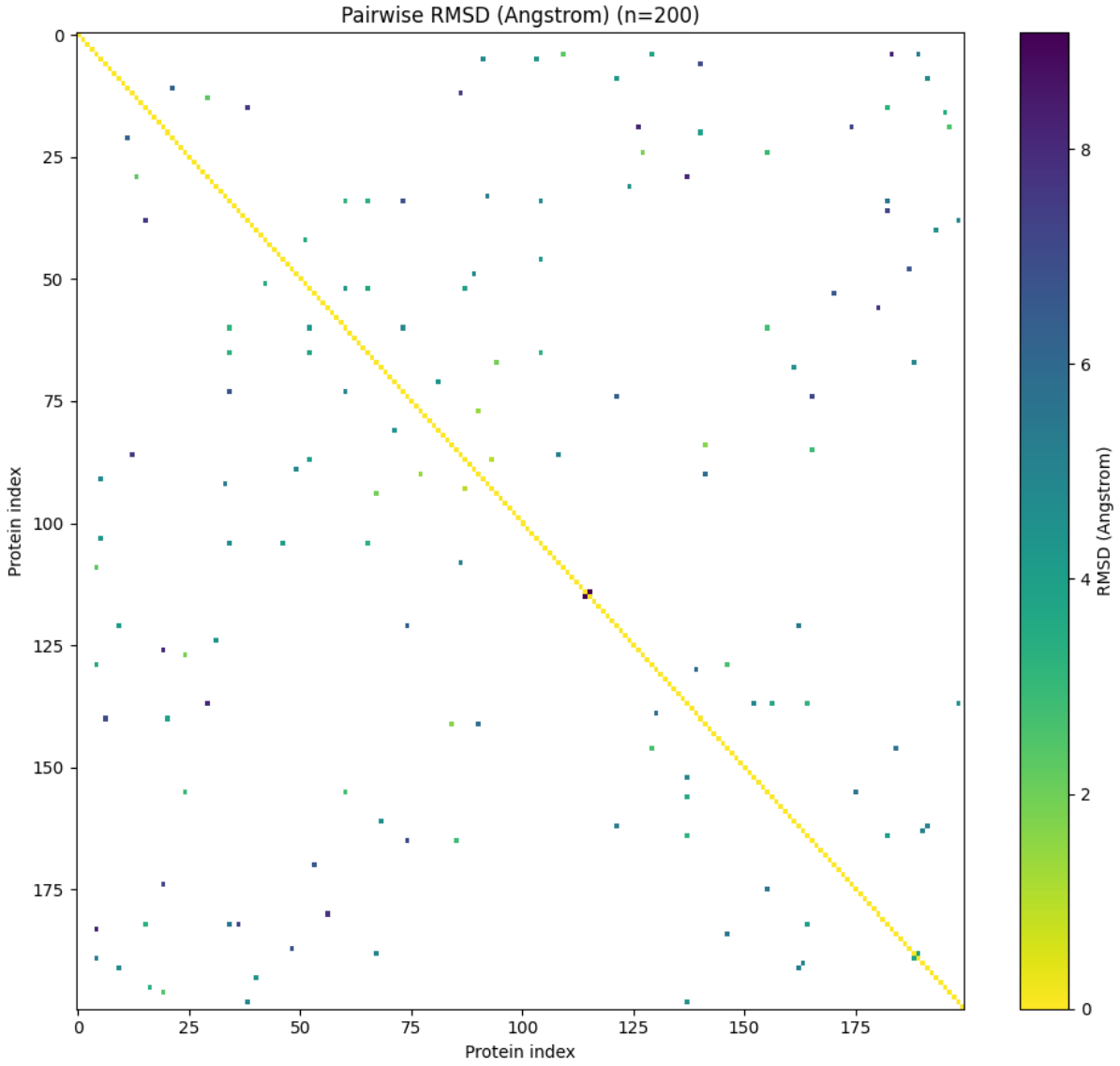

```
Length: mean=133, median=123, min=9, max=300
```

Pairwise heatmaps

Subsampled to 200 proteins for readability.

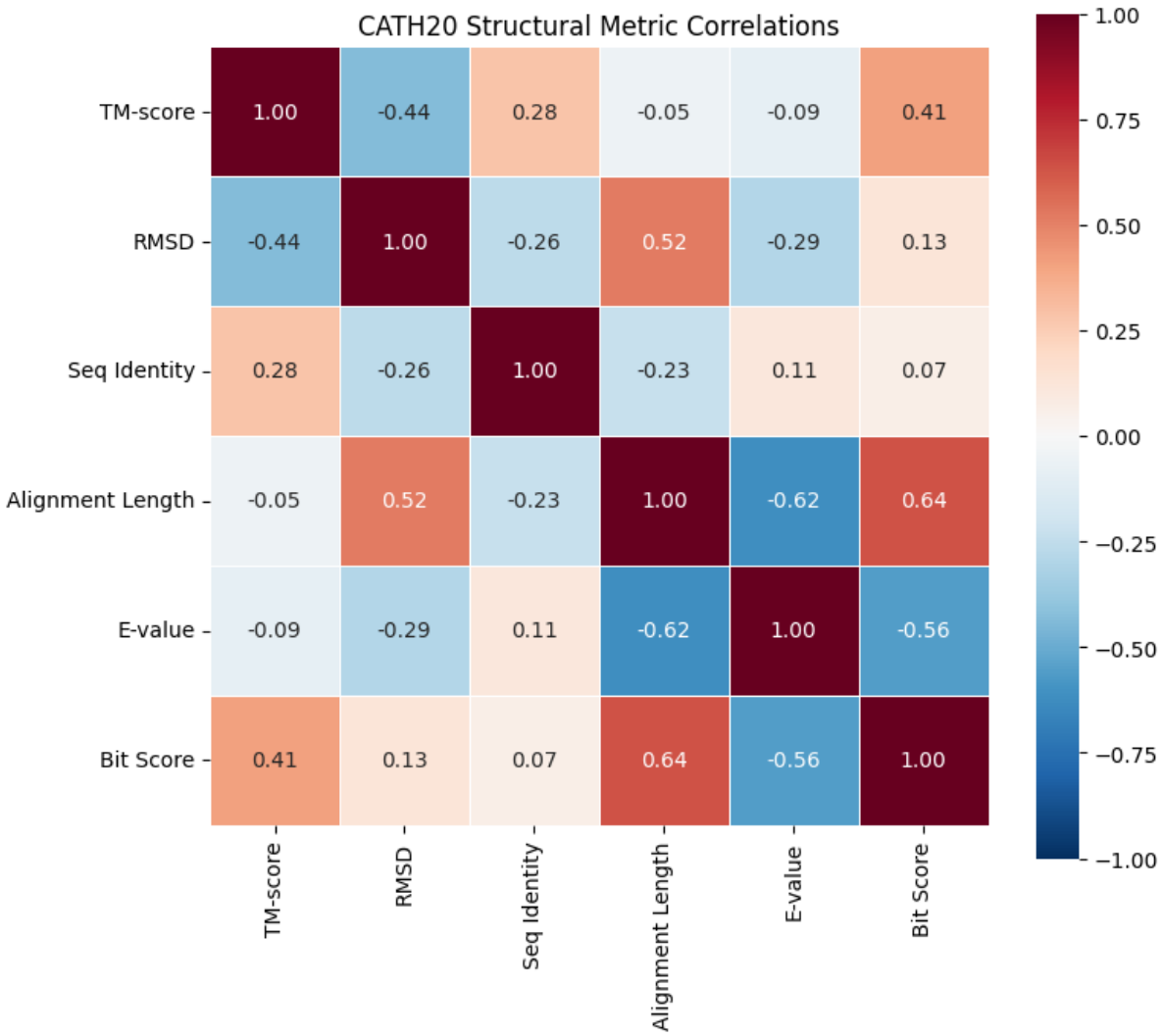
In [50]:
```
MAX_HEATMAP = 200
heatmap_proteins = all_proteins[:MAX_HEATMAP]

tm_matrix = build_pairwise_matrix(df, "alntmscore", heatmap_proteins)
plot_pairwise_heatmap(tm_matrix, heatmap_proteins, "TM-score", cmap="Y

rmsd_matrix = build_pairwise_matrix(df, "rmsd", heatmap_proteins)
plot_pairwise_heatmap(rmsd_matrix, heatmap_proteins, "RMSD (Angstrom)"
```


Pairwise TM-score (n=200)
Protein index
Protein index
TM-score
0
25
50
75
100
125
150
175
0.0
0.2
0.4
0.6
0.8
1.0



Metric correlations

In [51]:
```
_ = plot_correlation_heatmap(df, title="CATH20 Structural Metric Corre
```

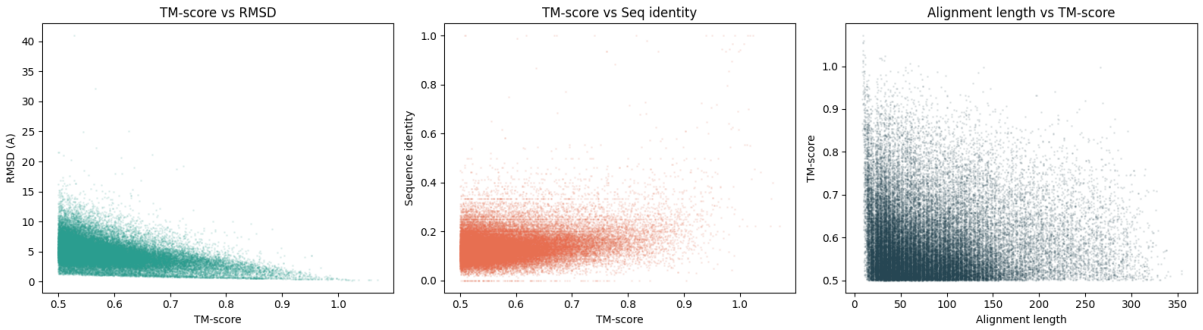


	TM-score	RMSD	Seq Identity	Alignment Length	E-value	Bit Score
TM-score	1.000	-0.437	0.285	-0.047	-0.089	0.412
RMSD	-0.437	1.000	-0.263	0.516	-0.291	0.130
Seq Identity	0.285	-0.263	1.000	-0.234	0.113	0.067
Alignment Length	-0.047	0.516	-0.234	1.000	-0.623	0.637
E-value	-0.089	-0.291	0.113	-0.623	1.000	-0.558
Bit Score	0.412	0.130	0.067	0.637	-0.558	1.000

Metric scatter plots

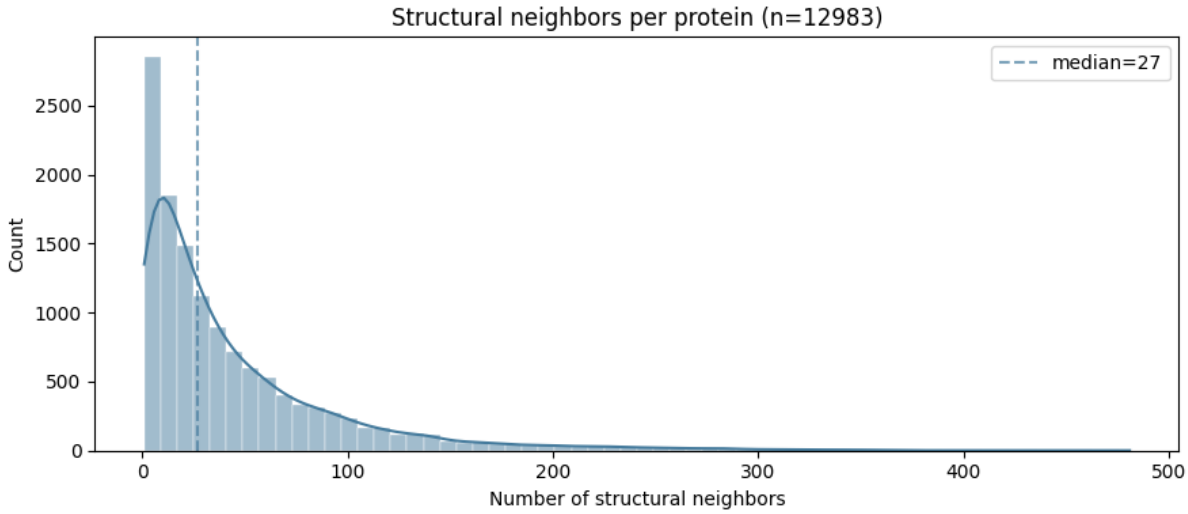
In [52]:
```
sample = non_self.sample(n=min(50000, len(non_self)), random_state=42)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))
```

```
axes[0].scatter(sample["alntmscore"], sample["rmsd"], s=1, alpha=0.1,
axes[0].set_xlabel("TM-score")
axes[0].set_ylabel("RMSD (A)")
axes[0].set_title("TM-score vs RMSD")

axes[1].scatter(sample["alntmscore"], sample["fident"], s=1, alpha=0.1
axes[1].set_xlabel("TM-score")
axes[1].set_ylabel("Sequence identity")
axes[1].set_title("TM-score vs Seq identity")

axes[2].scatter(sample["alnlen"], sample["alntmscore"], s=1, alpha=0.1
axes[2].set_xlabel("Alignment length")
axes[2].set_ylabel("TM-score")
axes[2].set_title("Alignment length vs TM-score")

fig.tight_layout()
plt.show()
```



Per-protein alignment connectivity

In [57]:
```
neighbors_per_protein = non_self.groupby("query")["target"].nunique()

fig, ax = plt.subplots(figsize=(9, 4))
sns.histplot(neighbors_per_protein, bins=60, color="#457b9d", edgecolo
ax.set_xlabel("Number of structural neighbors")
ax.set_ylabel("Count")
ax.set_title(f"Structural neighbors per protein (n={len(neighbors_per_
ax.axvline(neighbors_per_protein.median(), color="#457b9d", linestyle=
ax.legend()
fig.tight_layout()
plt.show()
```



Backrub Conformer Analysis (ai-cath subset)

Foldseek structural analysis and RMSD quality checks for backrub-generated conformers. Reproduces the analysis from `run_foldseek_backrub.sh`:

1. Foldseek all-vs-all on original structures only
2. Foldseek all-vs-all on originals + backrub conformers combined
3. RMSD analysis (backrub conformers vs their parent originals)

In [58]:
```
BACKRUB_BASE = Path("/Users/joycemo/Documents/PhD/Rotation3/dataset/ai
```

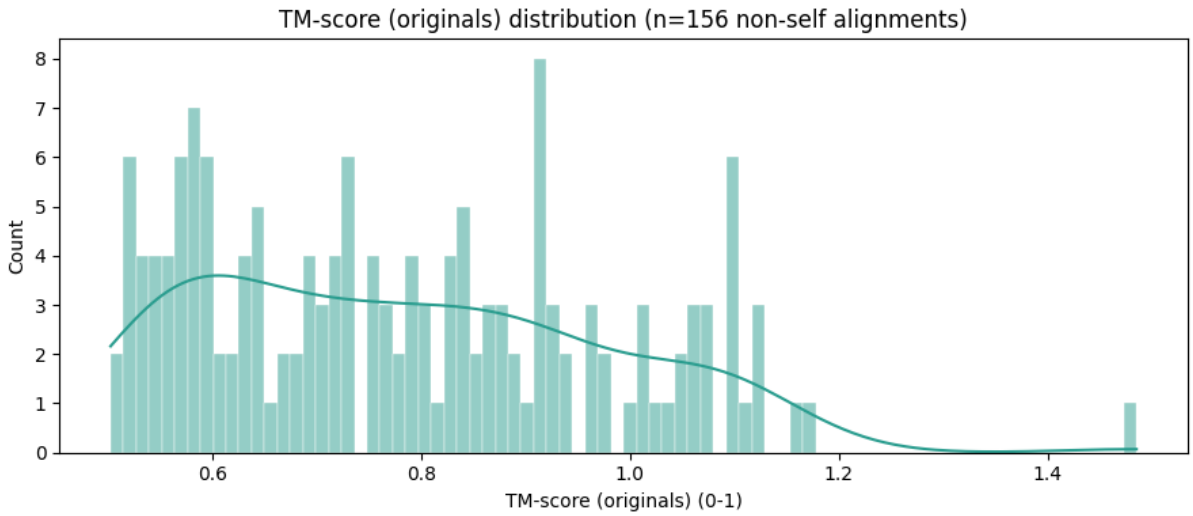
Foldseek on originals only

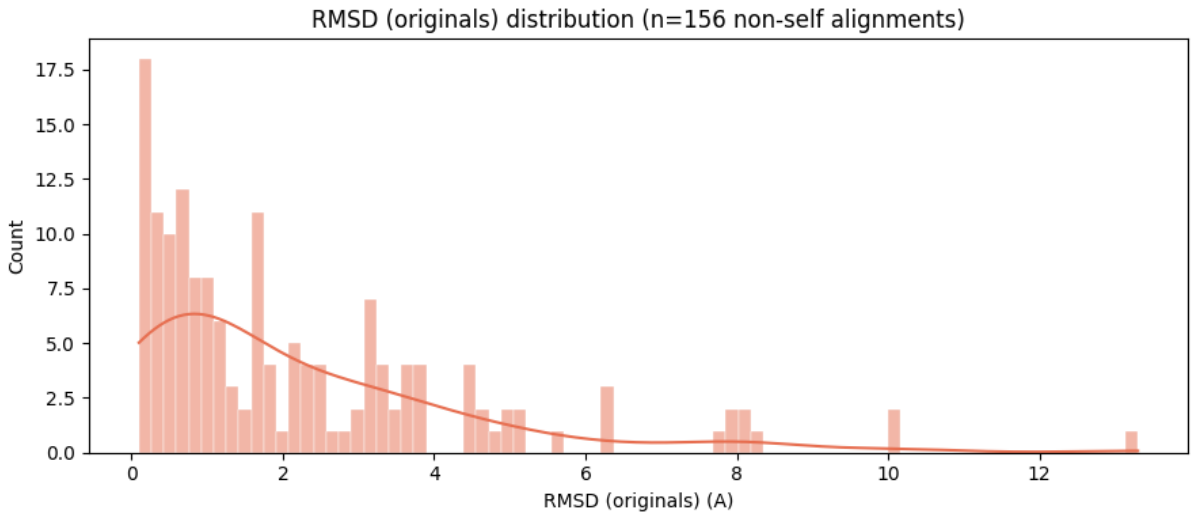
All-vs-all structural alignment of the 30 original ai-cath structures.

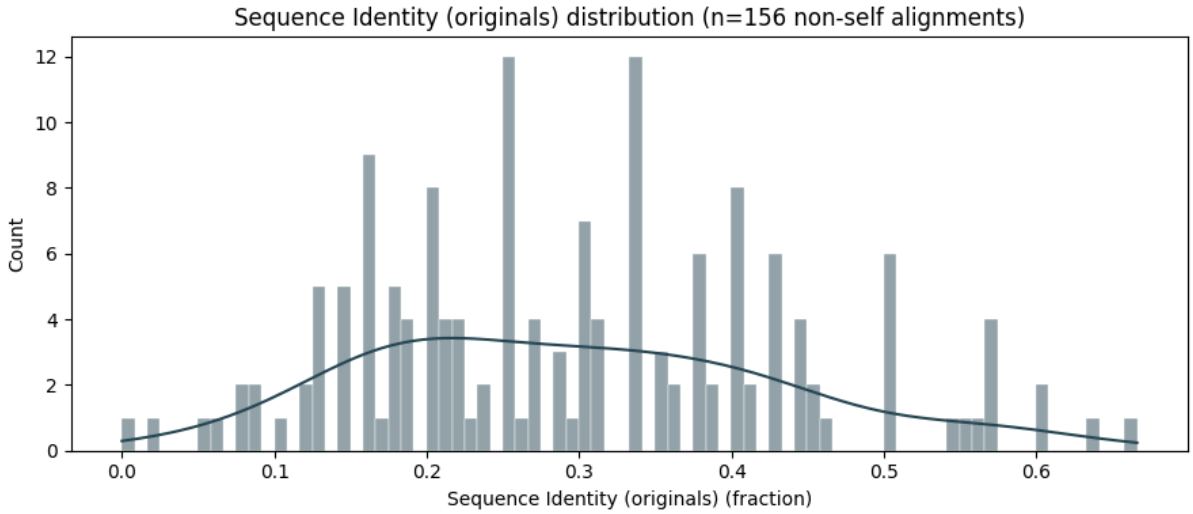
In [59]:
```
orig_df = load_foldseek_results(BACKRUB_BASE / "originals" / "foldseek
orig_proteins = sorted(orig_df["query"].unique())
orig_non_self = orig_df[orig_df["query"] != orig_df["target"]]

print(f"Originals: {len(orig_df)} alignments, {len(orig_proteins)} pro
print(f"Non-self: {len(orig_non_self)}")
print()

for col, label in [("alntmscore", "TM-score"), ("rmsd", "RMSD"), ("fid
    vals = orig_non_self[col].dropna()
    print(f"{label}: mean={vals.mean():.3f}, median={vals.median():.3f
          f"min={vals.min():.3f}, max={vals.max():.3f}")
```

```
Originals: 186 alignments, 30 proteins
Non-self: 156

TM-score: mean=0.782, median=0.760, min=0.503, max=1.485
RMSD: mean=2.229, median=1.561, min=0.098, max=13.300
Seq identity: mean=0.297, median=0.285, min=0.000, max=0.666
```

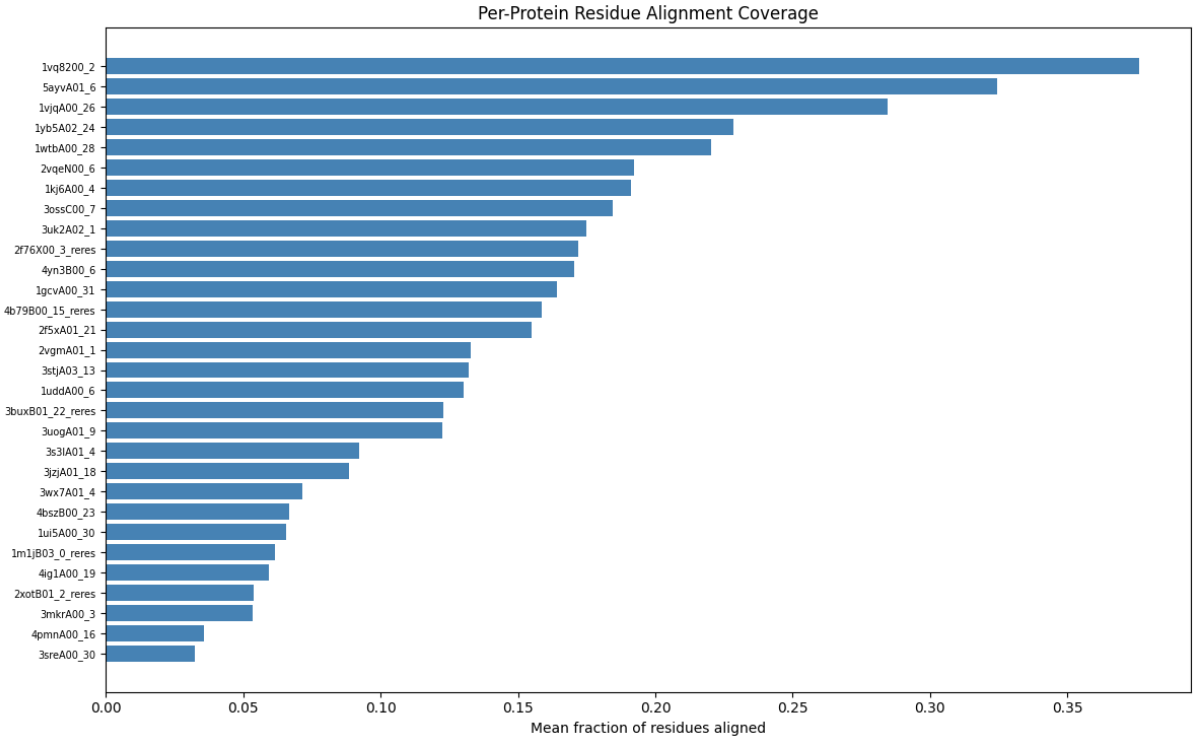
In [60]:
```
plot_metric_distribution(orig_df, "alntmscore", "TM-score (originals)"
plot_metric_distribution(orig_df, "rmsd", "RMSD (originals)", "A", col
plot_metric_distribution(orig_df, "fident", "Sequence Identity (origin
```







In [63]:
```
plot_residue_alignment_per_protein(orig_df, max_proteins=30)
```



Foldseek on originals + backrub conformers (combined)

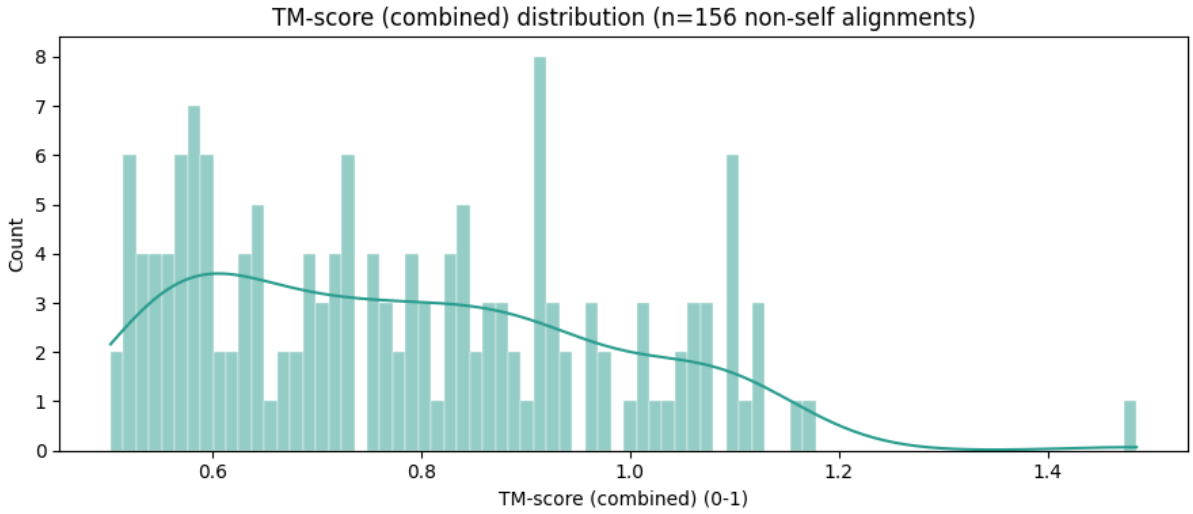
Shows how conformer ensembles cluster relative to their parent structures. Conformers from the same protein should cluster together if backrub preserves the fold.

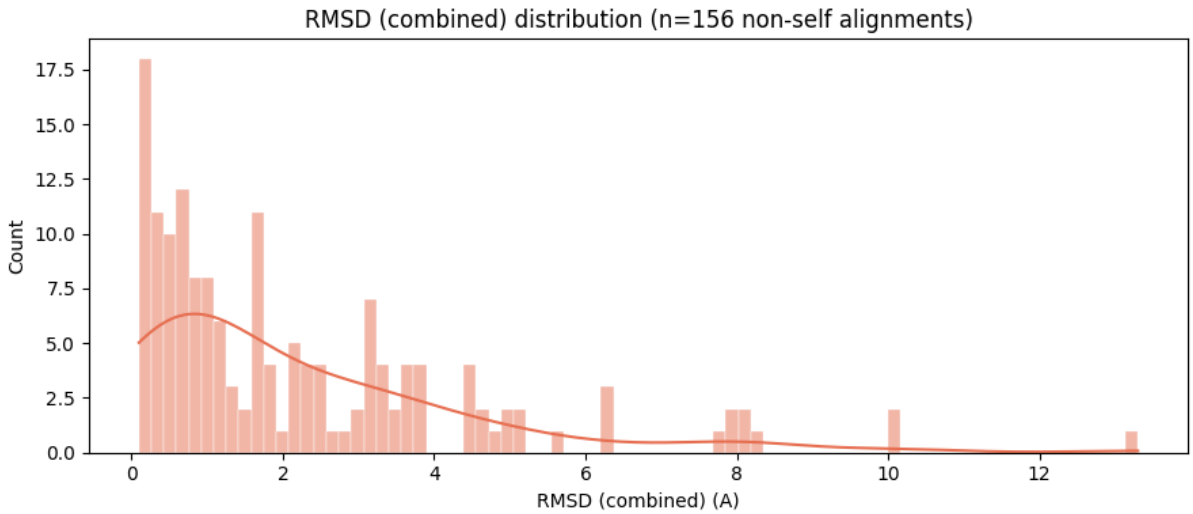
In [64]:
```
comb_df = load_foldseek_results(BACKRUB_BASE / "combined" / "foldseek_
comb_proteins = sorted(comb_df["query"].unique())
comb_non_self = comb_df[comb_df["query"] != comb_df["target"]]
```

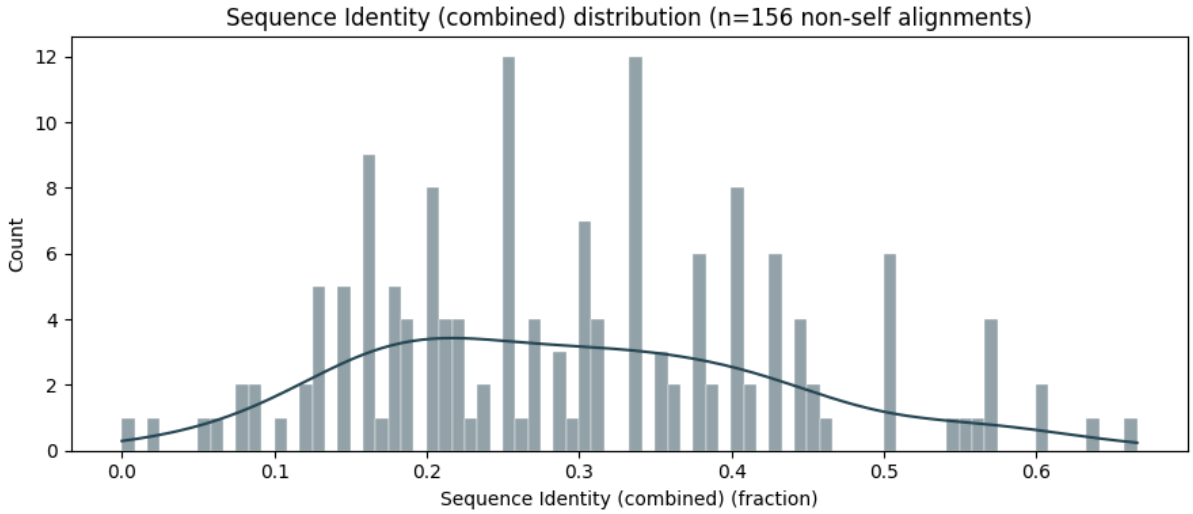
```
print(f"Combined: {len(comb_df)} alignments, {len(comb_proteins)} prot
print(f"Non-self: {len(comb_non_self)}")
print()

for col, label in [("alntmscore", "TM-score"), ("rmsd", "RMSD"), ("fid
    vals = comb_non_self[col].dropna()
    print(f"{label}: mean={vals.mean():.3f}, median={vals.median():.3f
          f"min={vals.min():.3f}, max={vals.max():.3f}")
```

```
Combined: 186 alignments, 30 proteins/conformers
Non-self: 156

TM-score: mean=0.782, median=0.760, min=0.503, max=1.485
RMSD: mean=2.229, median=1.561, min=0.098, max=13.300
Seq identity: mean=0.297, median=0.285, min=0.000, max=0.666
```

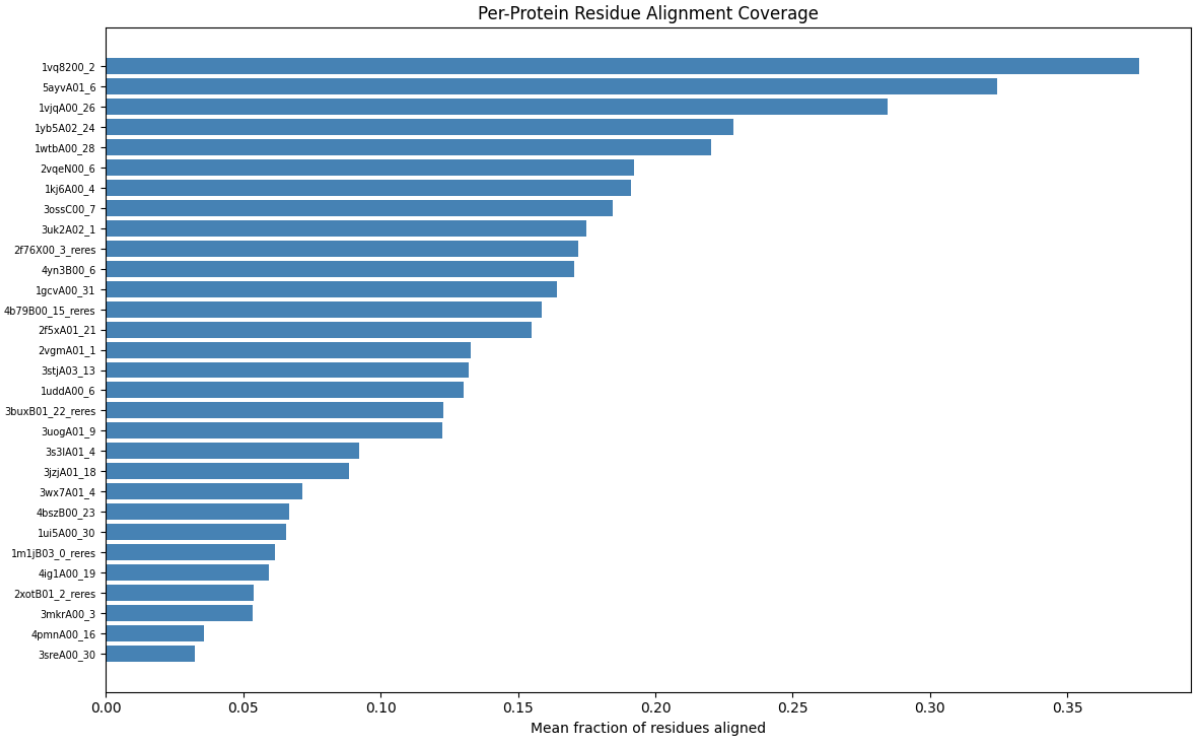
In [65]:
```
plot_metric_distribution(comb_df, "alntmscore", "TM-score (combined)",
plot_metric_distribution(comb_df, "rmsd", "RMSD (combined)", "A", colo
plot_metric_distribution(comb_df, "fident", "Sequence Identity (combin
```







In [67]:
```
plot_residue_alignment_per_protein(comb_df, max_proteins=50)
```



Backrub RMSD quality check

Kabsch RMSD and DRMSD between each backrub conformer and its parent original.

In [69]:
```
rmsd_df = pd.read_csv(BACKRUB_BASE / "rmsd" / "backrub_rmsd.csv")

print(f"Total conformer comparisons: {len(rmsd_df)}")
print(f"Unique proteins: {rmsd_df['protein'].nunique()}")
print()
print(f"RMSD:  mean={rmsd_df['rmsd'].mean():.3f} A, std={rmsd_df['rmsd
```

```
      f"min={rmsd_df['rmsd'].min():.3f}, max={rmsd_df['rmsd'].max():.3
print(f"DRMSD: mean={rmsd_df['drmsd'].mean():.3f} A, std={rmsd_df['drm
      f"min={rmsd_df['drmsd'].min():.3f}, max={rmsd_df['drmsd'].max():
```

```
Total conformer comparisons: 150
Unique proteins: 30

RMSD:  mean=2.350 A, std=4.259, min=0.436, max=24.260
DRMSD: mean=1.921 A, std=3.663, min=0.309, max=21.022
```

In [70]:
```
# Per-protein RMSD summary table
rmsd_summary = rmsd_df.groupby("protein")["rmsd"].agg(["mean", "std",
display(rmsd_summary.round(3))

print()

# Per-protein DRMSD summary table
drmsd_summary = rmsd_df.groupby("protein")["drmsd"].agg(["mean", "std"
display(drmsd_summary.round(3))
```

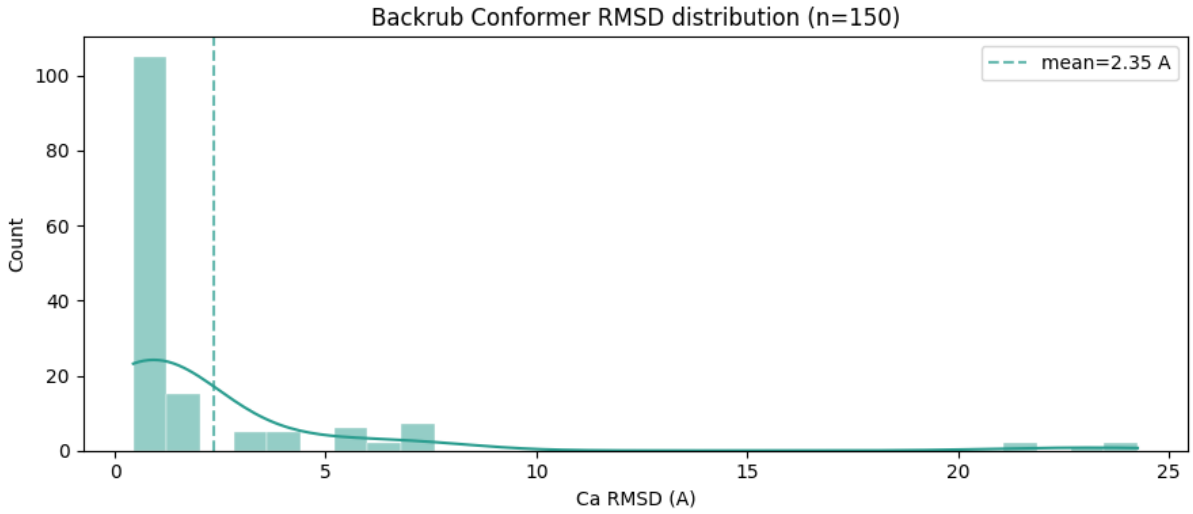
protein	mean	std	min	max	count
1gcvA00_31	0.578	0.000	0.578	0.578	5
1kj6A00_4	1.965	0.034	1.919	1.989	5
1m1jB03_0_reres	23.004	1.264	21.613	24.260	5
1uddA00_6	0.496	0.003	0.493	0.501	5
1ui5A00_30	5.215	0.003	5.214	5.221	5
1vjqA00_26	0.595	0.000	0.595	0.595	5
1vq8200_2	7.501	0.001	7.500	7.502	5
1wtbA00_28	0.828	0.011	0.821	0.848	5
1yb5A02_24	0.557	0.000	0.557	0.557	5
2f5xA01_21	1.114	0.004	1.107	1.116	5
2f76X00_3_reres	0.937	0.007	0.928	0.942	5
2vgmA01_1	6.618	0.469	5.873	7.024	5
2vqeN00_6	4.066	0.003	4.064	4.072	5
2xotB01_2_reres	0.938	0.010	0.933	0.955	5
3buxB01_22_reres	1.142	0.004	1.137	1.147	5
3jzjA01_18	0.663	0.000	0.663	0.663	5
3mkrA00_3	1.217	0.000	1.217	1.217	5

3ossC00_7	0.523	0.003	0.518	0.525	5
3s3lA01_4	1.008	0.002	1.004	1.009	5
3sreA00_30	0.933	0.002	0.931	0.936	5
3stjA03_13	0.588	0.004	0.580	0.590	5
3uk2A02_1	0.518	0.000	0.518	0.518	5
3uogA01_9	1.286	0.006	1.276	1.289	5
3wx7A01_4	1.560	0.021	1.536	1.576	5
4b79B00_15_reres	0.914	0.000	0.914	0.914	5
4bszB00_23	0.437	0.002	0.436	0.441	5
4ig1A00_19	0.758	0.005	0.755	0.767	5
4pmnA00_16	0.784	0.009	0.778	0.799	5
4yn3B00_6	3.201	0.001	3.199	3.202	5
5ayvA01_6	0.542	0.001	0.541	0.544	5

protein	mean	std	min	max	count
1gcvA00_31	0.479	0.001	0.479	0.480	5
1kj6A00_4	1.180	0.011	1.162	1.191	5
1m1jB03_0_reres	19.548	1.484	17.906	21.022	5
1uddA00_6	0.357	0.001	0.356	0.358	5
1ui5A00_30	4.053	0.007	4.050	4.065	5
1vjqA00_26	0.459	0.000	0.459	0.459	5
1vq8200_2	6.798	0.002	6.796	6.799	5
1wtbA00_28	0.559	0.004	0.556	0.566	5
1yb5A02_24	0.368	0.000	0.368	0.368	5
2f5xA01_21	0.890	0.002	0.887	0.891	5
2f76X00_3_reres	0.618	0.007	0.608	0.623	5
2vgmA01_1	5.915	0.443	5.225	6.335	5
2vqeN00_6	3.269	0.001	3.268	3.271	5
2xotB01_2_reres	0.784	0.010	0.780	0.802	5
3buxB01_22_reres	0.778	0.002	0.776	0.780	5
3jzjA01_18	0.535	0.000	0.535	0.535	5

3mkrA00_3	0.710	0.000	0.710	0.710	5
3ossC00_7	0.463	0.005	0.454	0.465	5
3s3lA01_4	0.805	0.003	0.799	0.806	5
3sreA00_30	0.707	0.001	0.706	0.709	5
3stjA03_13	0.396	0.001	0.395	0.398	5
3uk2A02_1	0.357	0.001	0.355	0.357	5
3uogA01_9	0.940	0.003	0.935	0.944	5
3wx7A01_4	1.448	0.020	1.426	1.463	5
4b79B00_15_reres	0.874	0.000	0.874	0.874	5
4bszB00_23	0.309	0.001	0.309	0.312	5
4ig1A00_19	0.543	0.008	0.539	0.558	5
4pmnA00_16	0.573	0.009	0.566	0.587	5
4yn3B00_6	2.522	0.001	2.520	2.523	5
5ayvA01_6	0.398	0.000	0.398	0.399	5

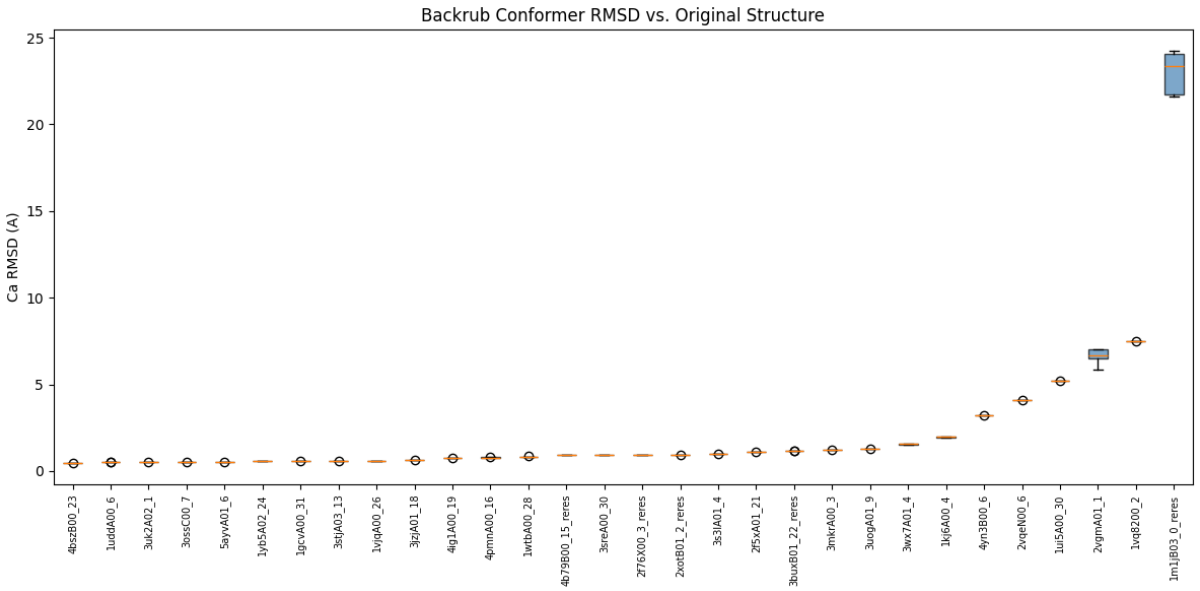
In [71]:
```
# RMSD histogram
fig, ax = plt.subplots(figsize=(9, 4))
sns.histplot(rmsd_df["rmsd"].dropna(), bins=30, color="#2a9d8f", edgec
             linewidth=0.3, kde=True, ax=ax)
ax.axvline(rmsd_df["rmsd"].mean(), color="#2a9d8f", linestyle="--", al
           label=f"mean={rmsd_df['rmsd'].mean():.2f} A")
ax.set_xlabel("Ca RMSD (A)")
ax.set_ylabel("Count")
ax.set_title(f"Backrub Conformer RMSD distribution (n={len(rmsd_df)})"
ax.legend()
fig.tight_layout()
plt.show()

print(f"RMSD: mean={rmsd_df['rmsd'].mean():.3f}, median={rmsd_df['rmsd
      f"min={rmsd_df['rmsd'].min():.3f}, max={rmsd_df['rmsd'].max():.3
```




```
RMSD: mean=2.350, median=0.933, min=0.436, max=24.260
```

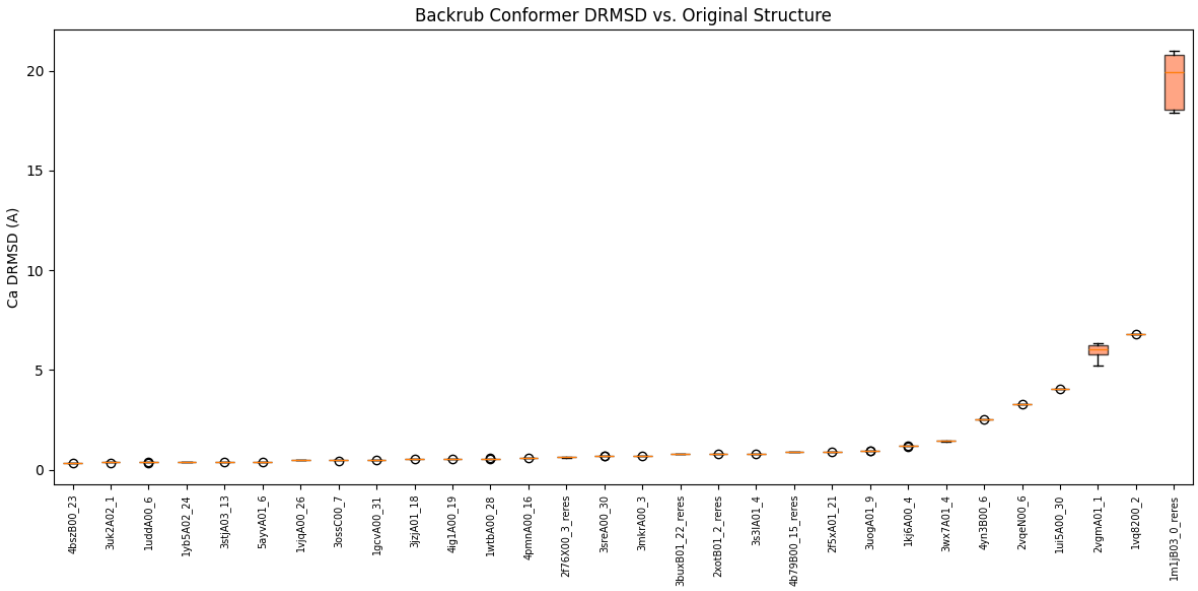
In [72]:
```
# RMSD box plot per protein
proteins_sorted = rmsd_df.groupby("protein")["rmsd"].mean().sort_value

fig, ax = plt.subplots(figsize=(max(8, len(proteins_sorted) * 0.4), 6)
df_plot = rmsd_df.set_index("protein").loc[proteins_sorted].reset_inde
bp = ax.boxplot(
    [df_plot[df_plot["protein"] == p]["rmsd"].values for p in proteins
    labels=proteins_sorted,
    patch_artist=True,
)
for patch in bp["boxes"]:
    patch.set_facecolor("steelblue")
    patch.set_alpha(0.7)

ax.set_xticklabels(proteins_sorted, rotation=90, fontsize=7)
ax.set_ylabel("Ca RMSD (A)")
ax.set_title("Backrub Conformer RMSD vs. Original Structure")
fig.tight_layout()
plt.show()
```

```
/var/folders/17/p_x2_shj61dfmmkzqsf2dfq40000gn/T/ipykernel_94846/122761
8835.py:6: MatplotlibDeprecationWarning: The 'labels' parameter of boxp
lot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  bp = ax.boxplot(
```

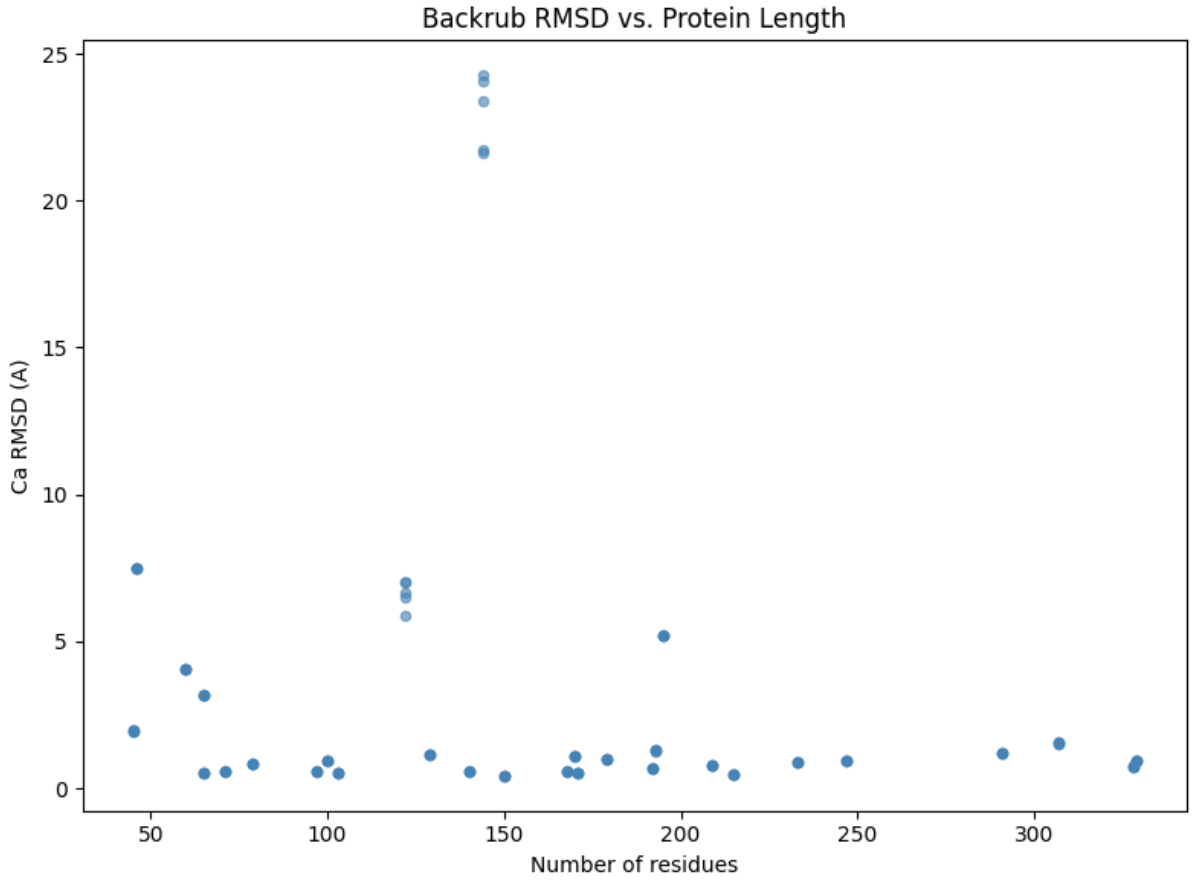


In [73]:
```
# DRMSD box plot per protein
proteins_sorted_d = rmsd_df.groupby("protein")["drmsd"].mean().sort_va

fig, ax = plt.subplots(figsize=(max(8, len(proteins_sorted_d) * 0.4),
df_plot_d = rmsd_df.set_index("protein").loc[proteins_sorted_d].reset_
bp = ax.boxplot(
    [df_plot_d[df_plot_d["protein"] == p]["drmsd"].values for p in pro
    labels=proteins_sorted_d,
    patch_artist=True,
)
for patch in bp["boxes"]:
    patch.set_facecolor("coral")
    patch.set_alpha(0.7)

ax.set_xticklabels(proteins_sorted_d, rotation=90, fontsize=7)
ax.set_ylabel("Ca DRMSD (A)")
ax.set_title("Backrub Conformer DRMSD vs. Original Structure")
fig.tight_layout()
plt.show()
```

```
/var/folders/17/p_x2_shj61dfmmkzqsf2dfq40000gn/T/ipykernel_94846/358213
2127.py:6: MatplotlibDeprecationWarning: The 'labels' parameter of boxp
lot() has been renamed 'tick_labels' since Matplotlib 3.9; support for
the old name will be dropped in 3.11.
  bp = ax.boxplot(
```



In [74]:
```
# RMSD vs protein length scatter
fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(rmsd_df["n_residues"], rmsd_df["rmsd"], s=20, alpha=0.6, c=
ax.set_xlabel("Number of residues")
ax.set_ylabel("Ca RMSD (A)")
ax.set_title("Backrub RMSD vs. Protein Length")
fig.tight_layout()
plt.show()
```



Protpardelle representation UMAPs

use `umap_representations.py` on two sets of extracted embeddings to visualize how the model's internal representations organize residues.

These embeddings/representations will be used for representation alignment. it will alos be used to compare how well a VAE's latent representations will compare to these other embeddings -> questions of what is the best way to represent protein structure data for diffusion models?

UMAP of march_rep_test embeddings

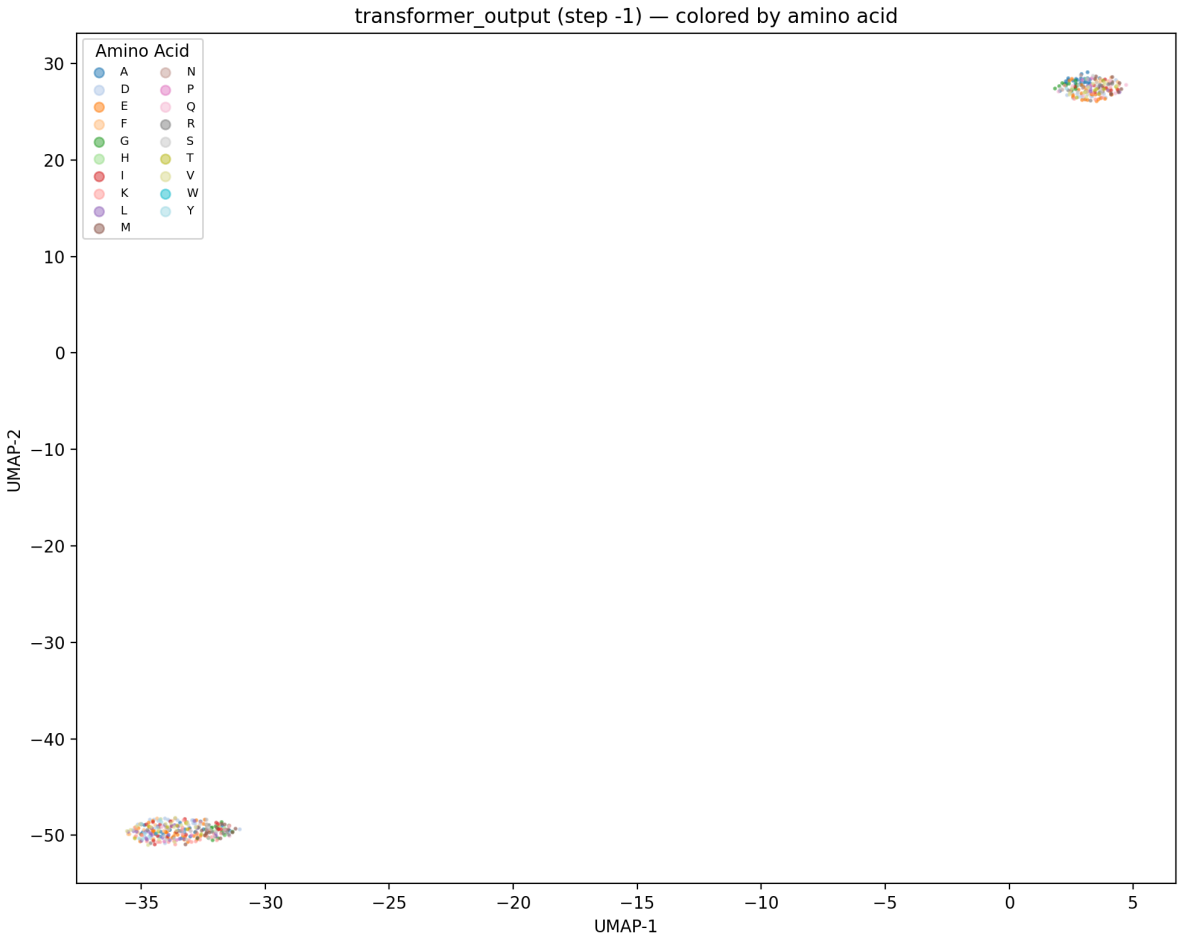
Representations from:
`/Users/joycemo/Documents/GitHub/protpardelle-1c/results/march_rep_test`

In [75]:
```
import subprocess, sys

UMAP_SCRIPT = "/Users/joycemo/Documents/GitHub/protpardelle-1c/represe

result = subprocess.run(
    [
        sys.executable, UMAP_SCRIPT,
        "--rep-dir", "/Users/joycemo/Documents/GitHub/protpardelle-1c/
        "--output-dir", "output/umap_march_rep_test",
        "--module", "transformer_output",
        "--step", "-1",
        "--max-residues", "200000",
    ],
    capture_output=True, text=True,
)
print(result.stdout)
if result.stderr:
    print(result.stderr)
```

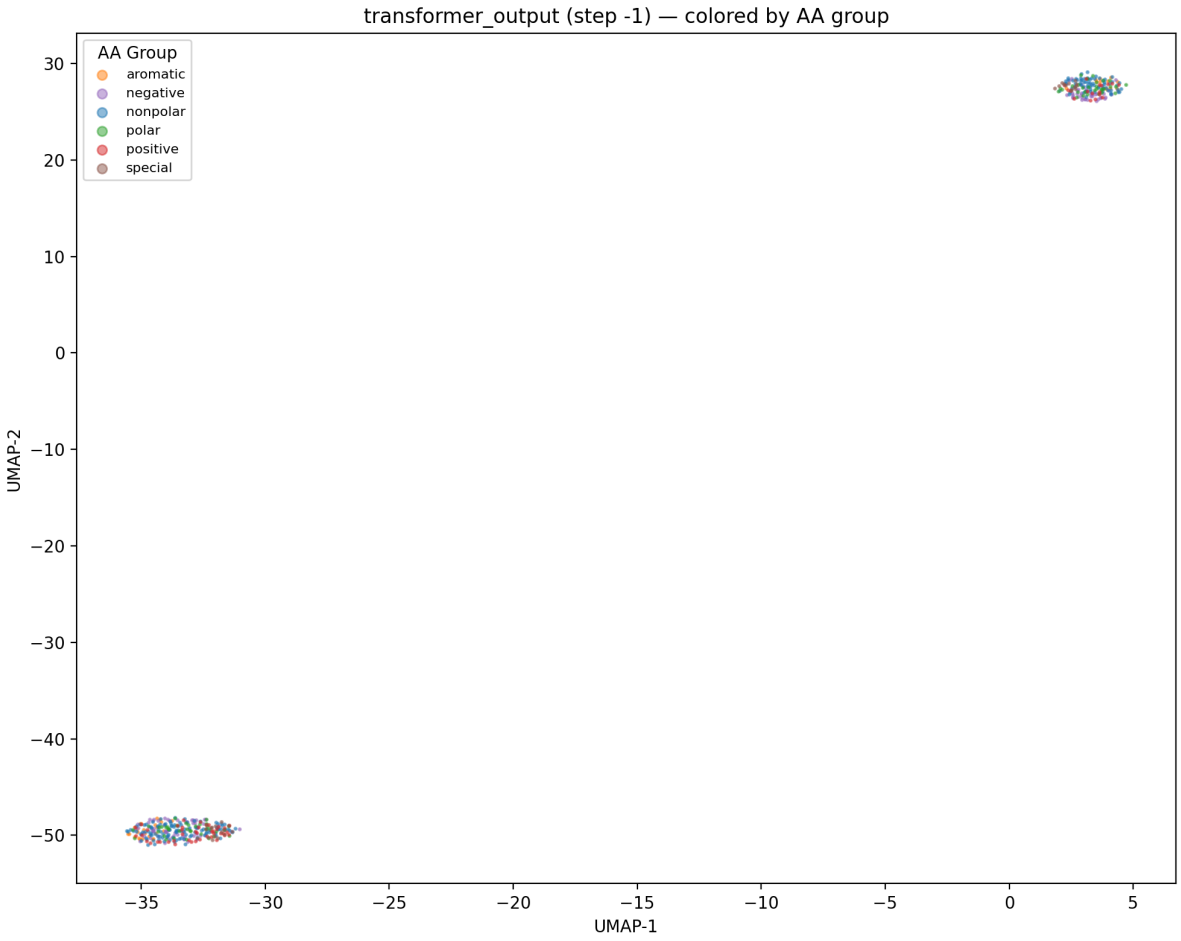
```
Found 5 structures in /Users/joycemo/Documents/GitHub/protpardelle-1c/r
esults/march_rep_test/per_structure
Loaded 470 residue embeddings (256D) from 5 structures
Running UMAP (n_neighbors=100, min_dist=0.3, metric=euclidean, n=470)
...
Saved 7 UMAP plots to output/umap_march_rep_test/

/Users/joycemo/miniconda3/envs/rotation3-local/lib/python3.11/site-pack
ages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
/Users/joycemo/Documents/GitHub/protpardelle-1c/representation_extracti
on/umap_representations.py:271: MatplotlibDeprecationWarning: The get_c
map function was deprecated in Matplotlib 3.7 and will be removed in 3.
11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cm
ap()`` or ``pyplot.get_cmap()`` instead.
  cmap = plt.cm.get_cmap("tab20", len(aa_list))
```

In [76]:
```
from IPython.display import Image, display

umap_dir = Path("output/umap_march_rep_test")
for fig_path in sorted(umap_dir.glob("*.png")):
    print(fig_path.name)
    display(Image(filename=str(fig_path), width=600))
```

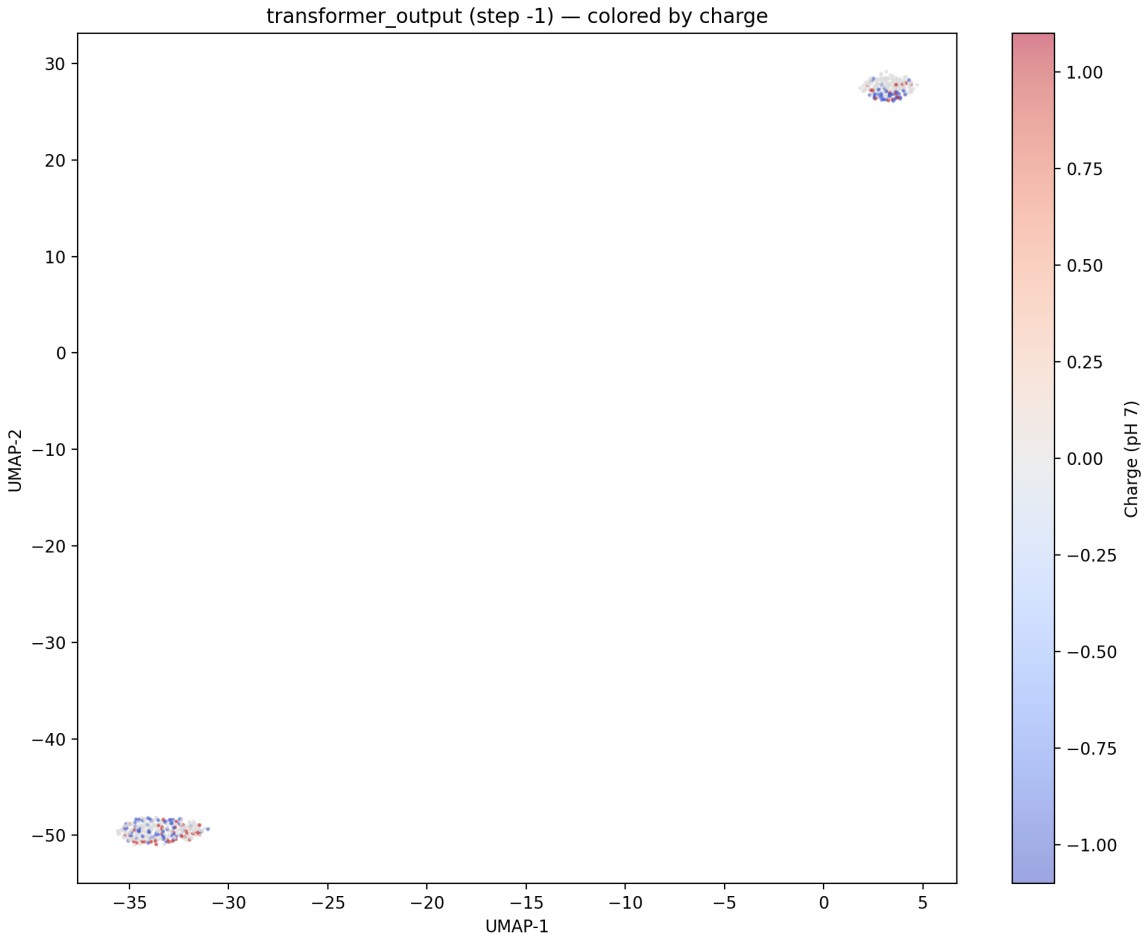
```
transformer_output_step-1_by_aa.png
```



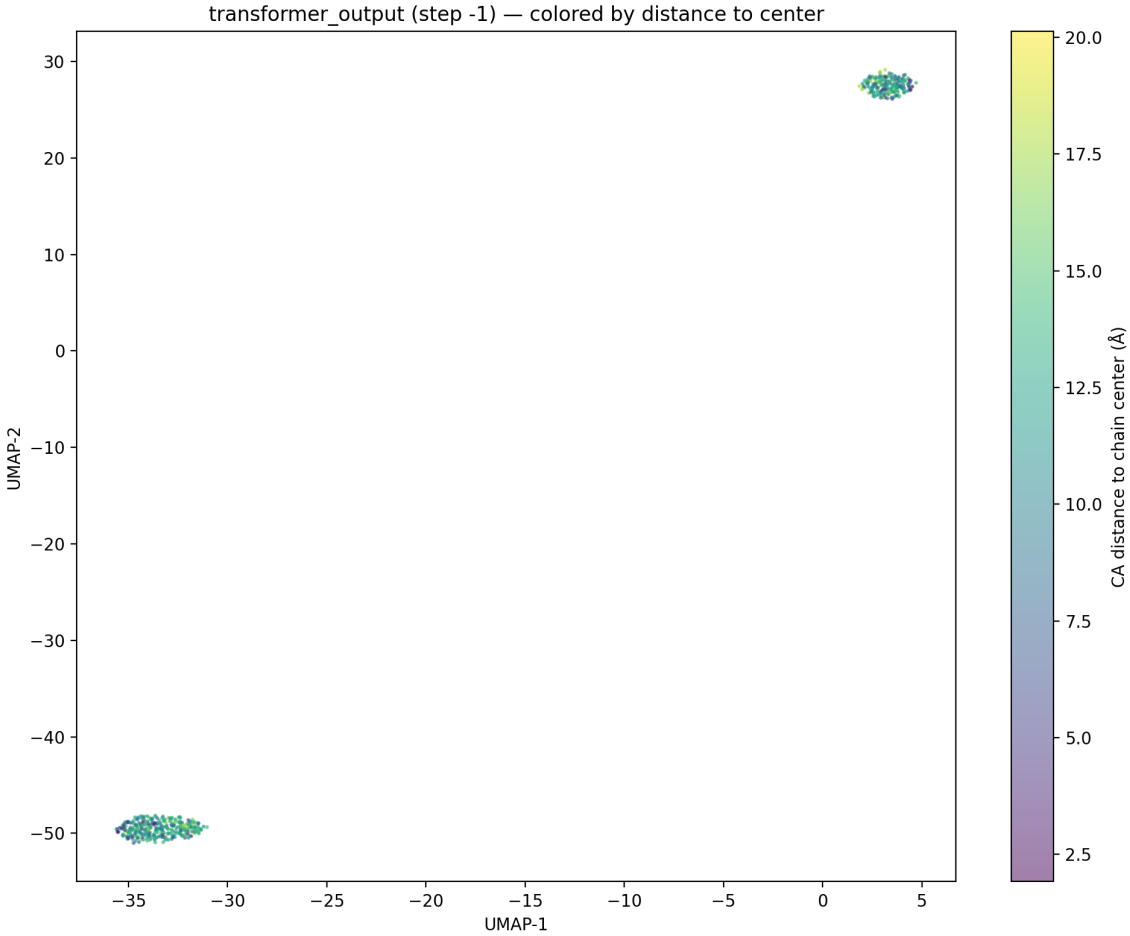
transformer_output_step−1_by_aa_group.png



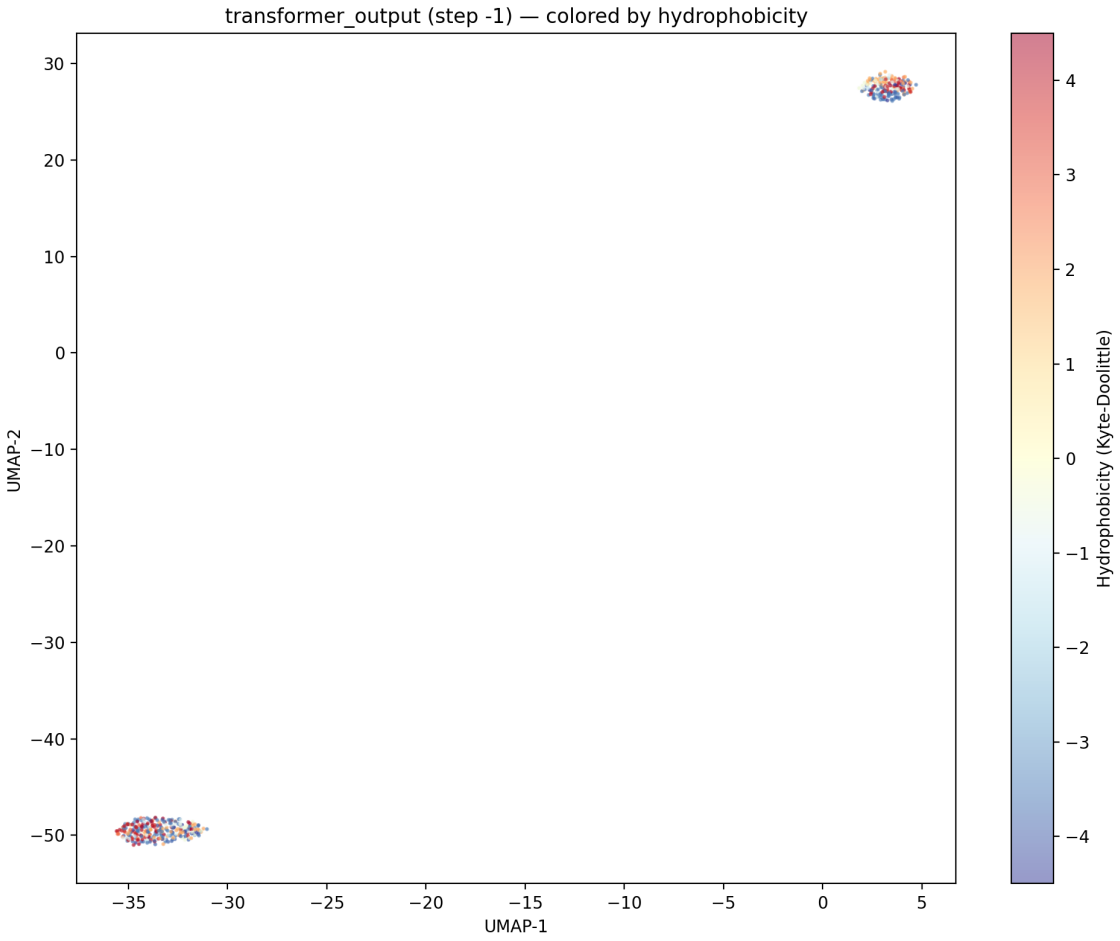
transformer_output_step−1_by_charge.png



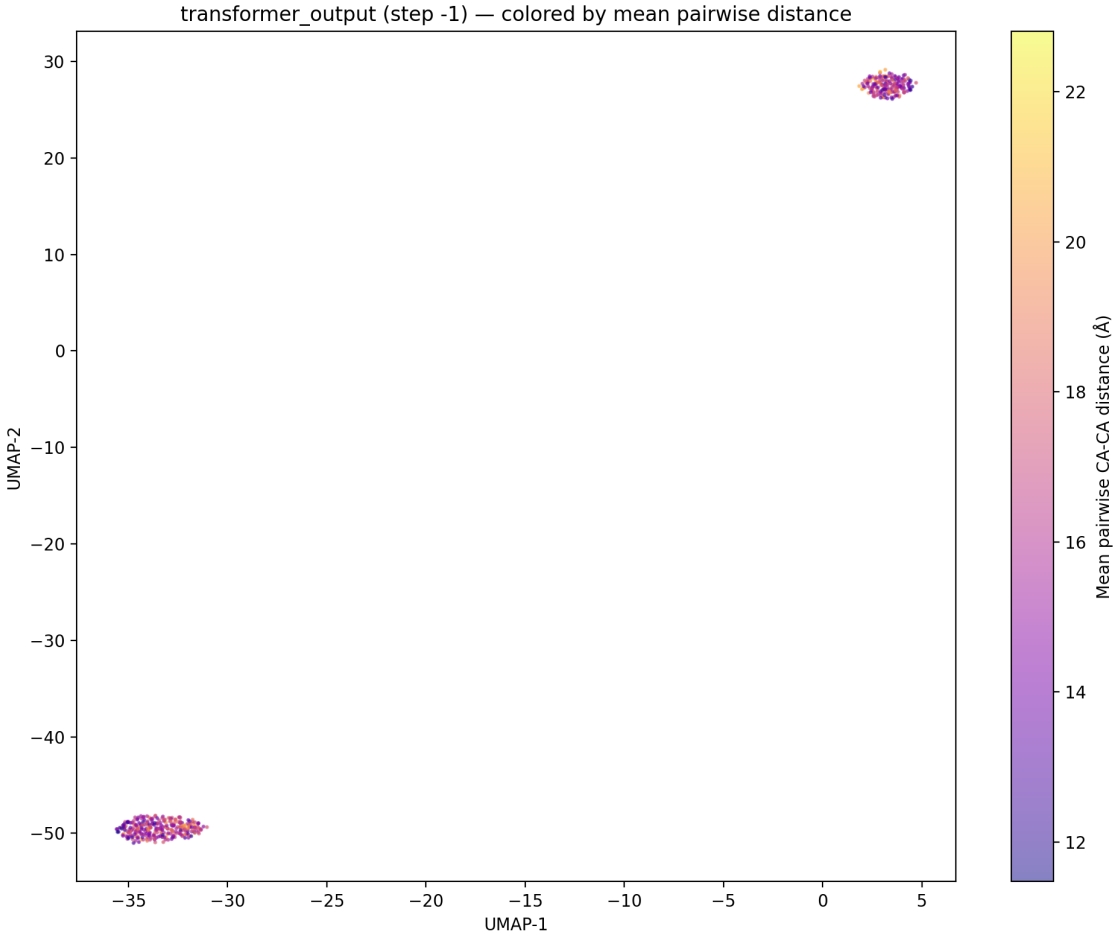
transformer_output_step−1_by_dist_to_center.png



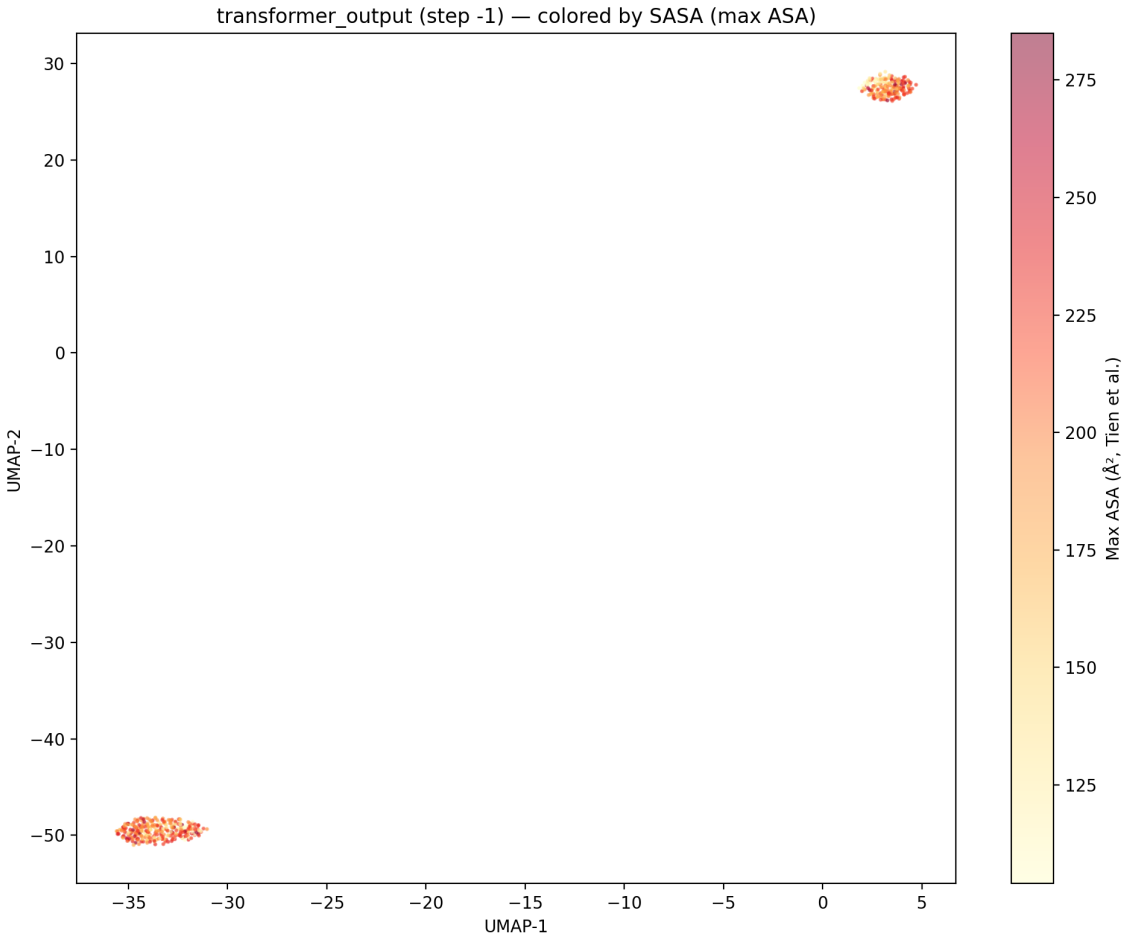
transformer_output_step−1_by_hydrophobicity.png



transformer_output_step−1_by_mean_pw_dist.png



transformer_output_step−1_by_sasa.png



UMAP of CATH20 Protpardelle representations

Representations from:
`/Users/joycemo/Documents/PhD/Rotation3/cath20_representations/pro`

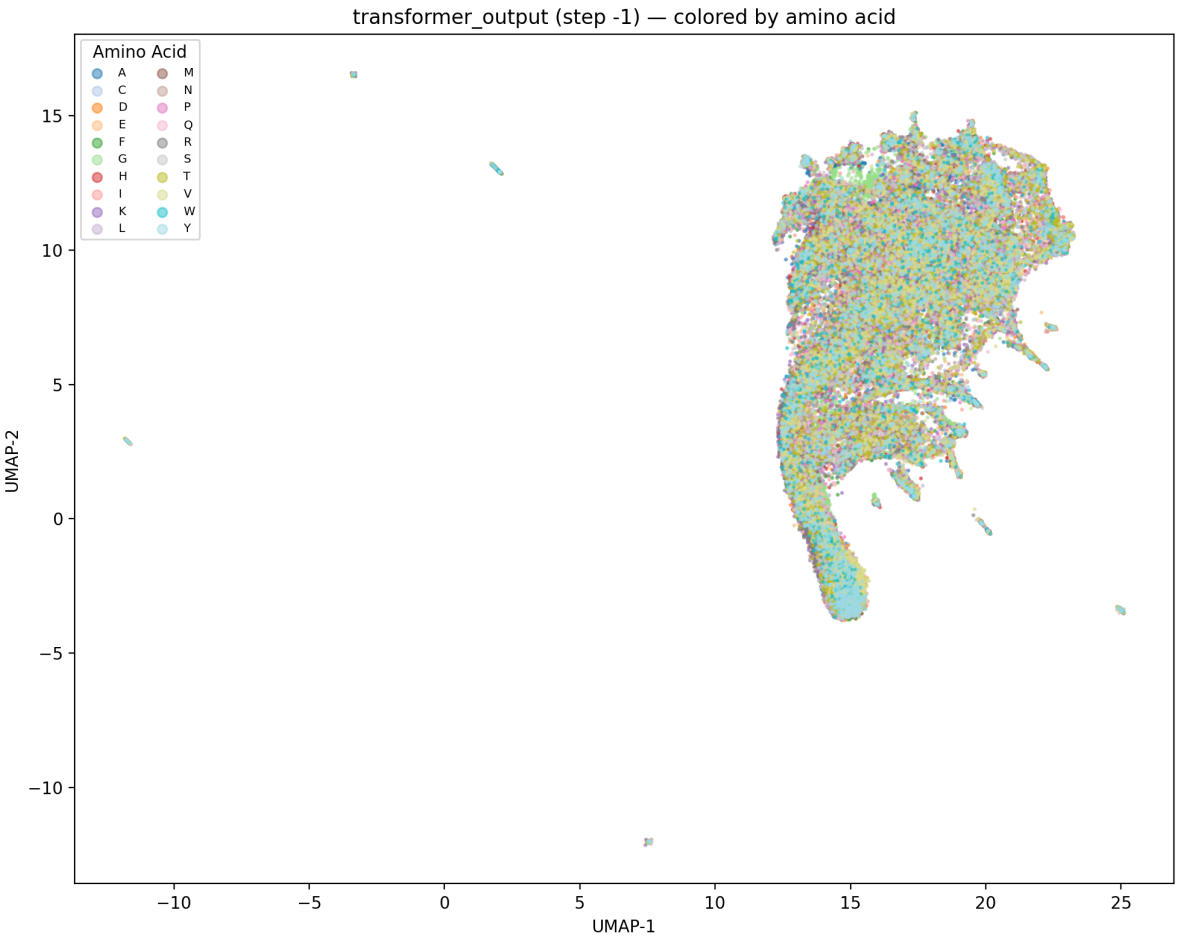
This takes a long time to run and so I will just run this on Wynton.

In [80]:
```
result = subprocess.run(
    [
        sys.executable, UMAP_SCRIPT,
        "--rep-dir", "/Users/joycemo/Documents/PhD/Rotation3/cath20_re
        "--output-dir", "output/umap_cath20_protpardelle",
        "--module", "transformer_output",
        "--step", "-1",
        "--max-residues", "200000",
    ],
    capture_output=True, text=True,
)
print(result.stdout)
if result.stderr:
    print(result.stderr)
```

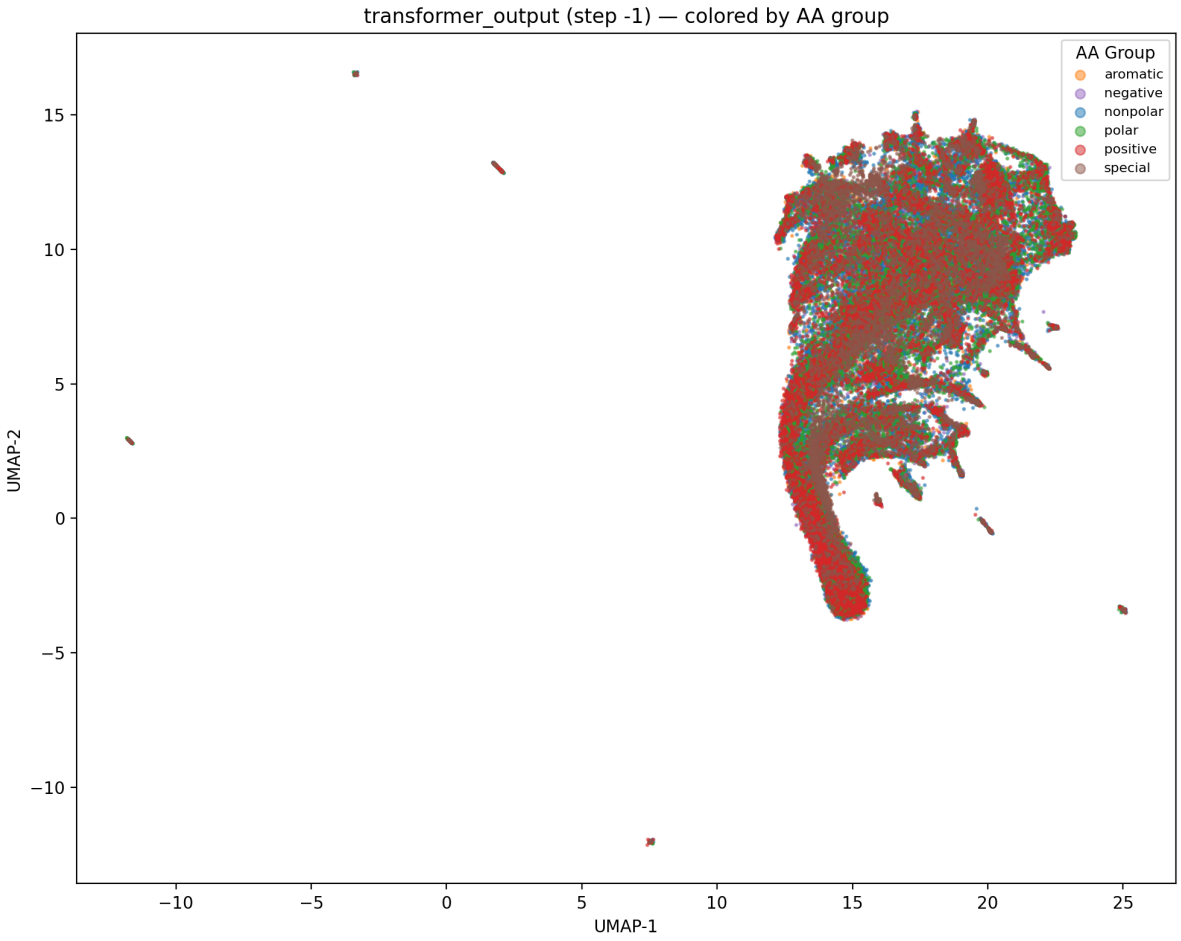
```
Found 566 structures in /Users/joycemo/Documents/PhD/Rotation3/cath20_r
epresentations/protpardelle/per_structure
Loaded 78262 residue embeddings (256D) from 565 structures
Running UMAP (n_neighbors=100, min_dist=0.3, metric=euclidean, n=78262)
...
Saved 7 UMAP plots to output/umap_cath20_protpardelle/

/Users/joycemo/miniconda3/envs/rotation3-local/lib/python3.11/site-pack
ages/umap/umap_.py:1952: UserWarning: n_jobs value 1 overridden to 1 by
setting random_state. Use no seed for parallelism.
  warn(
/Users/joycemo/Documents/GitHub/protpardelle-1c/representation_extracti
on/umap_representations.py:271: MatplotlibDeprecationWarning: The get_c
map function was deprecated in Matplotlib 3.7 and will be removed in 3.
11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cm
ap()`` or ``pyplot.get_cmap()`` instead.
  cmap = plt.cm.get_cmap("tab20", len(aa_list))
```

In [81]:
```
umap_dir = Path("output/umap_cath20_protpardelle")
for fig_path in sorted(umap_dir.glob("*.png")):
    print(fig_path.name)
    display(Image(filename=str(fig_path), width=600))
```

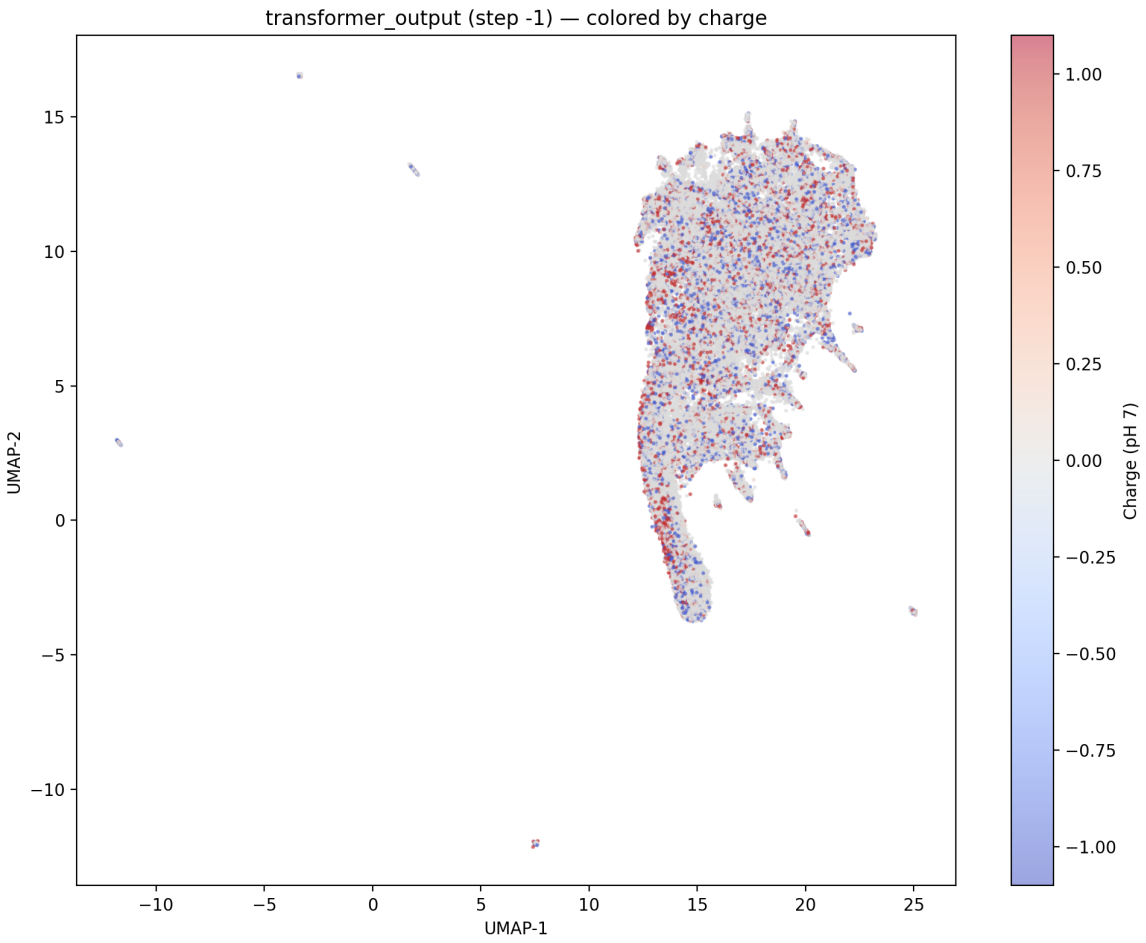
transformer_output_step-1_by_aa.png



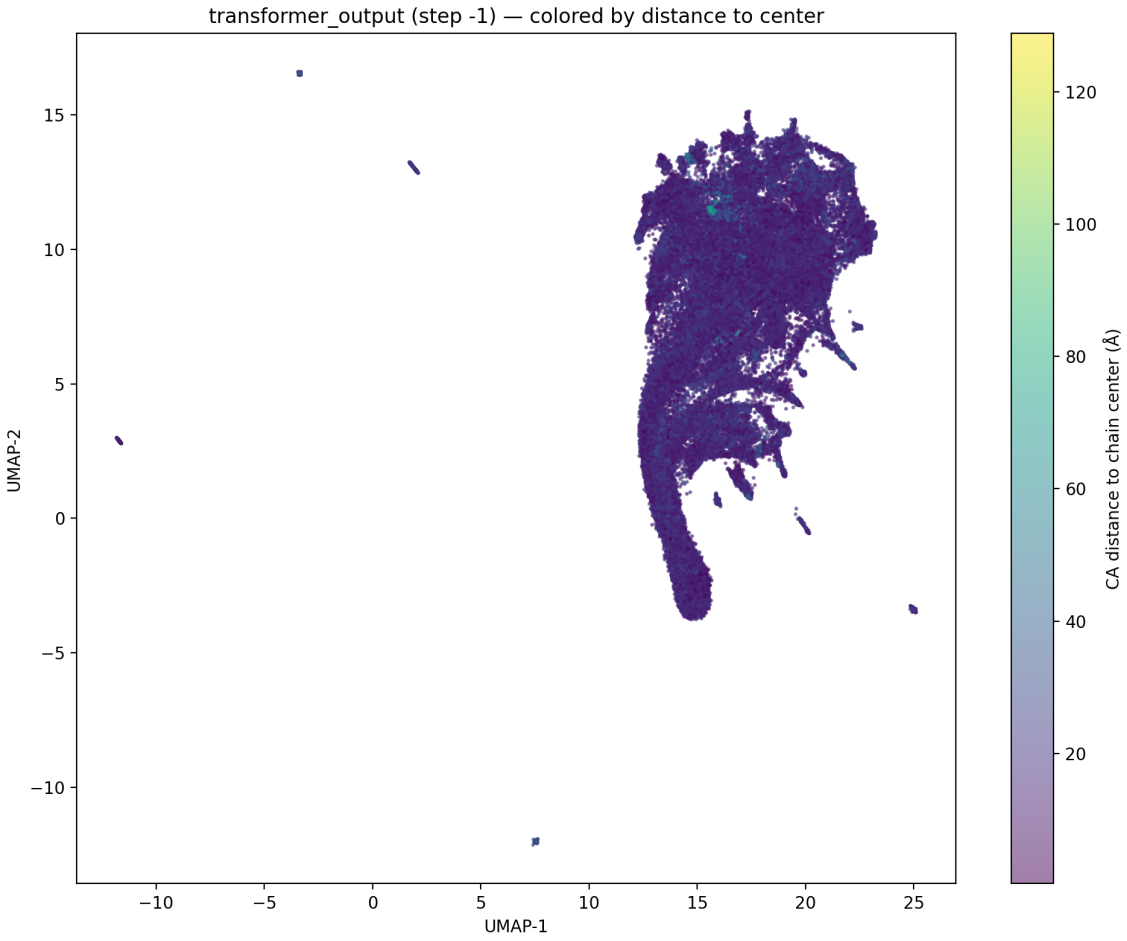
transformer_output_step-1_by_aa_group.png



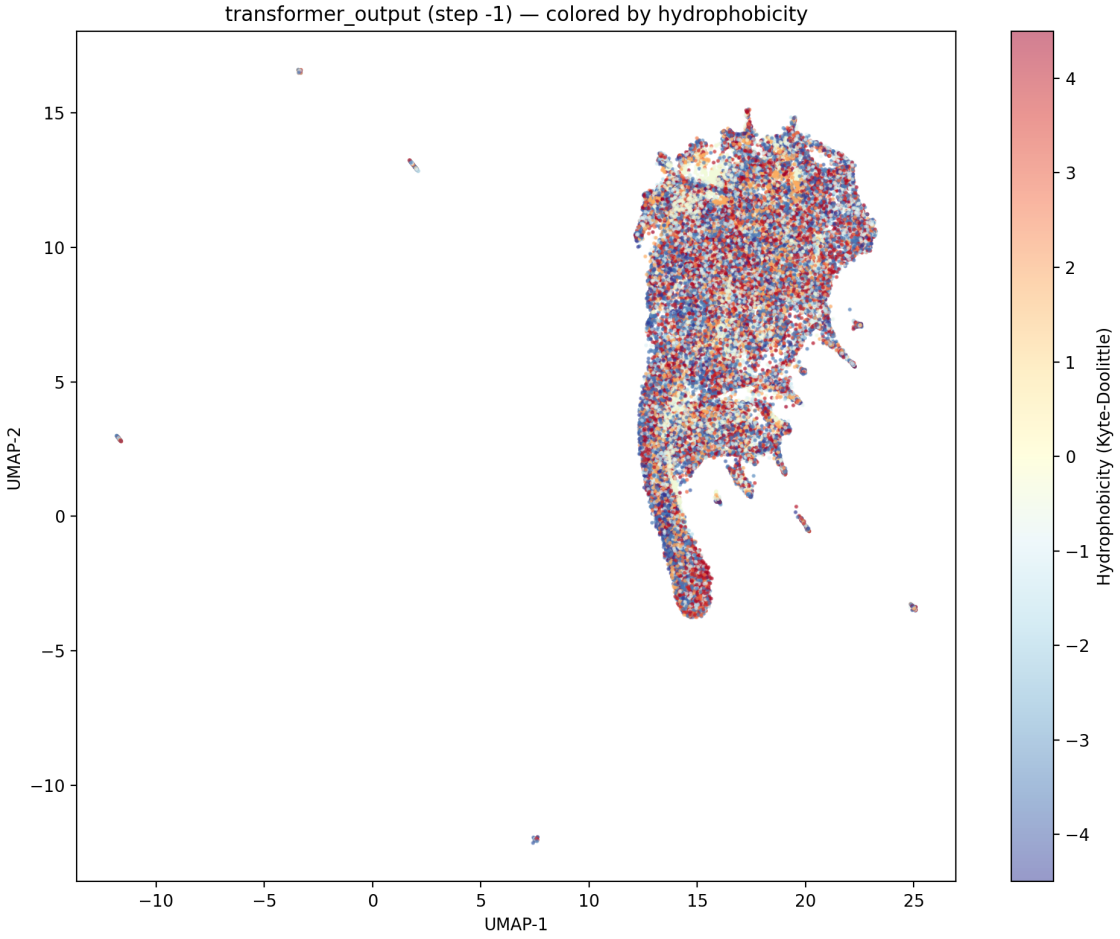
transformer_output_step−1_by_charge.png



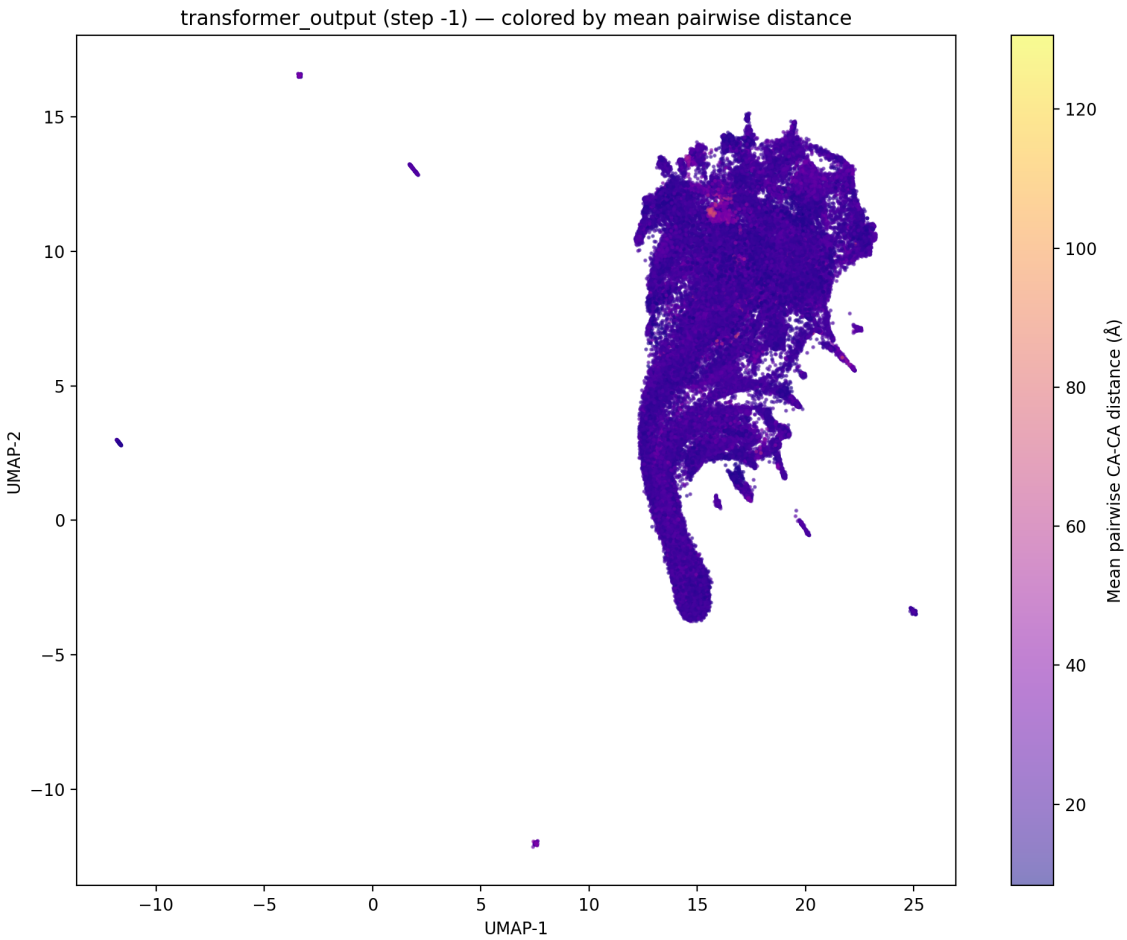
transformer_output_step−1_by_dist_to_center.png



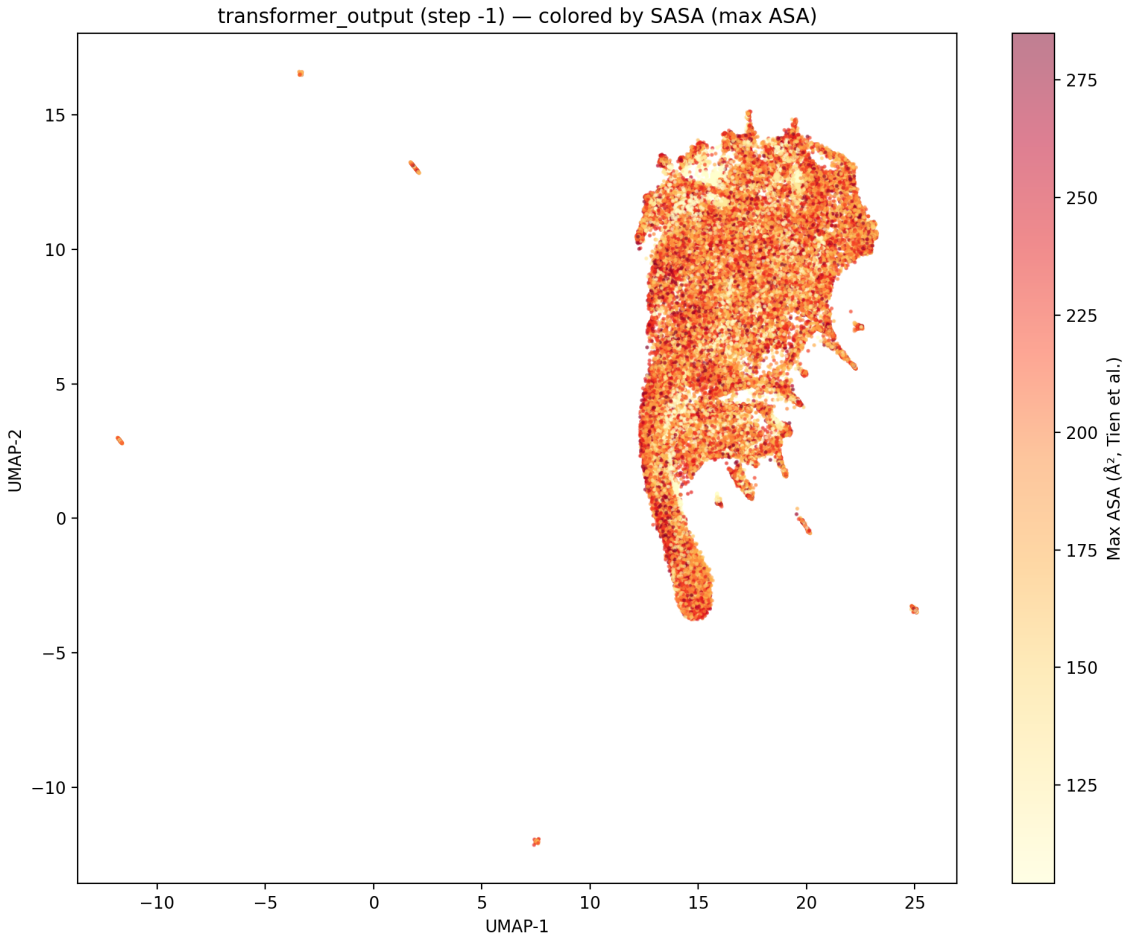
transformer_output_step−1_by_hydrophobicity.png



transformer_output_step−1_by_mean_pw_dist.png



transformer_output_step−1_by_sasa.png



In [84]:
```
!pip install nbconvert
!jupyter nbconvert --to html cath20_analysis.ipynb
```

```
Collecting nbconvert
  Downloading nbconvert-7.17.0-py3-none-any.whl.metadata (8.4 kB)
Collecting beautifulsoup4 (from nbconvert)
  Using cached beautifulsoup4-4.14.3-py3-none-any.whl.metadata (3.8 kB)
Collecting bleach!=5.0.0 (from bleach[css]!=5.0.0->nbconvert)
  Downloading bleach-6.3.0-py3-none-any.whl.metadata (31 kB)
Collecting defusedxml (from nbconvert)
  Downloading defusedxml-0.7.1-py2.py3-none-any.whl.metadata (32 kB)
Requirement already satisfied: jinja2>=3.0 in /Users/joycemo/miniconda
3/envs/rotation3-local/lib/python3.11/site-packages (from nbconvert) (
3.1.6)
Requirement already satisfied: jupyter-core>=4.7 in /Users/joycemo/mini
conda3/envs/rotation3-local/lib/python3.11/site-packages (from nbconver
t) (5.9.1)
Collecting jupyterlab-pygments (from nbconvert)
  Downloading jupyterlab_pygments-0.3.0-py3-none-any.whl.metadata (4.4
kB)
Requirement already satisfied: markupsafe>=2.0 in /Users/joycemo/minico
nda3/envs/rotation3-local/lib/python3.11/site-packages (from nbconvert)
(3.0.3)
Collecting mistune<4,>=2.0.3 (from nbconvert)
  Downloading mistune-3.2.0-py3-none-any.whl.metadata (1.9 kB)
```

```
Collecting nbclient>=0.5.0 (from nbconvert)
  Downloading nbclient-0.10.4-py3-none-any.whl.metadata (8.3 kB)
Collecting nbformat>=5.7 (from nbconvert)
  Downloading nbformat-5.10.4-py3-none-any.whl.metadata (3.6 kB)
Requirement already satisfied: packaging in /Users/joycemo/miniconda3/e
nvs/rotation3-local/lib/python3.11/site-packages (from nbconvert) (26.
0)
Collecting pandocfilters>=1.4.1 (from nbconvert)
  Downloading pandocfilters-1.5.1-py2.py3-none-any.whl.metadata (9.0 k
B)
Requirement already satisfied: pygments>=2.4.1 in /Users/joycemo/minico
nda3/envs/rotation3-local/lib/python3.11/site-packages (from nbconvert)
(2.20.0)
Requirement already satisfied: traitlets>=5.1 in /Users/joycemo/minicon
da3/envs/rotation3-local/lib/python3.11/site-packages (from nbconvert)
(5.14.3)
Collecting webencodings (from bleach!=5.0.0->bleach[css]!=5.0.0->nbconv
ert)
  Downloading webencodings-0.5.1-py2.py3-none-any.whl.metadata (2.1 kB)
Collecting tinycss2<1.5,>=1.1.0 (from bleach[css]!=5.0.0->nbconvert)
  Downloading tinycss2-1.4.0-py3-none-any.whl.metadata (3.0 kB)
Requirement already satisfied: platformdirs>=2.5 in /Users/joycemo/mini
conda3/envs/rotation3-local/lib/python3.11/site-packages (from jupyter-
core>=4.7->nbconvert) (4.9.4)
Requirement already satisfied: jupyter-client>=6.1.12 in /Users/joycem
o/miniconda3/envs/rotation3-local/lib/python3.11/site-packages (from nb
client>=0.5.0->nbconvert) (8.8.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /Users/joycem
o/miniconda3/envs/rotation3-local/lib/python3.11/site-packages (from ju
pyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (2.9.0.post0)
Requirement already satisfied: pyzmq>=25.0 in /Users/joycemo/miniconda
3/envs/rotation3-local/lib/python3.11/site-packages (from jupyter-clien
t>=6.1.12->nbclient>=0.5.0->nbconvert) (27.1.0)
Requirement already satisfied: tornado>=6.4.1 in /Users/joycemo/minicon
da3/envs/rotation3-local/lib/python3.11/site-packages (from jupyter-cli
ent>=6.1.12->nbclient>=0.5.0->nbconvert) (6.5.5)
Collecting fastjsonschema>=2.15 (from nbformat>=5.7->nbconvert)
  Downloading fastjsonschema-2.21.2-py3-none-any.whl.metadata (2.3 kB)
Collecting jsonschema>=2.6 (from nbformat>=5.7->nbconvert)
  Using cached jsonschema-4.26.0-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: attrs>=22.2.0 in /Users/joycemo/minicond
a3/envs/rotation3-local/lib/python3.11/site-packages (from jsonschema>=
2.6->nbformat>=5.7->nbconvert) (26.1.0)
Collecting jsonschema-specifications>=2023.03.6 (from jsonschema>=2.6->
nbformat>=5.7->nbconvert)
  Using cached jsonschema_specifications-2025.9.1-py3-none-any.whl.meta
data (2.9 kB)
Collecting referencing>=0.28.4 (from jsonschema>=2.6->nbformat>=5.7->nb
convert)
  Using cached referencing-0.37.0-py3-none-any.whl.metadata (2.8 kB)
Collecting rpds-py>=0.25.0 (from jsonschema>=2.6->nbformat>=5.7->nbconv
ert)
```

```
  Using cached rpds_py-0.30.0-cp311-cp311-macosx_11_0_arm64.whl.metadat
a (4.1 kB)
Requirement already satisfied: six>=1.5 in /Users/joycemo/miniconda3/en
vs/rotation3-local/lib/python3.11/site-packages (from python-dateutil>=
2.8.2->jupyter-client>=6.1.12->nbclient>=0.5.0->nbconvert) (1.17.0)
Requirement already satisfied: typing-extensions>=4.4.0 in /Users/joyce
mo/miniconda3/envs/rotation3-local/lib/python3.11/site-packages (from r
eferencing>=0.28.4->jsonschema>=2.6->nbformat>=5.7->nbconvert) (4.15.0)
Collecting soupsieve>=1.6.1 (from beautifulsoup4->nbconvert)
  Downloading soupsieve-2.8.3-py3-none-any.whl.metadata (4.6 kB)
Downloading nbconvert-7.17.0-py3-none-any.whl (261 kB)
Downloading mistune-3.2.0-py3-none-any.whl (53 kB)
Downloading bleach-6.3.0-py3-none-any.whl (164 kB)
Downloading tinycss2-1.4.0-py3-none-any.whl (26 kB)
Downloading nbclient-0.10.4-py3-none-any.whl (25 kB)
Downloading nbformat-5.10.4-py3-none-any.whl (78 kB)
Downloading fastjsonschema-2.21.2-py3-none-any.whl (24 kB)
Using cached jsonschema-4.26.0-py3-none-any.whl (90 kB)
Using cached jsonschema_specifications-2025.9.1-py3-none-any.whl (18 k
B)
Downloading pandocfilters-1.5.1-py2.py3-none-any.whl (8.7 kB)
Using cached referencing-0.37.0-py3-none-any.whl (26 kB)
Using cached rpds_py-0.30.0-cp311-cp311-macosx_11_0_arm64.whl (359 kB)
Downloading webencodings-0.5.1-py2.py3-none-any.whl (11 kB)
Using cached beautifulsoup4-4.14.3-py3-none-any.whl (107 kB)
Downloading soupsieve-2.8.3-py3-none-any.whl (37 kB)
Downloading defusedxml-0.7.1-py2.py3-none-any.whl (25 kB)
Downloading jupyterlab_pygments-0.3.0-py3-none-any.whl (15 kB)
Installing collected packages: webencodings, fastjsonschema, tinycss2,
soupsieve, rpds-py, pandocfilters, mistune, jupyterlab-pygments, defuse
dxml, bleach, referencing, beautifulsoup4, jsonschema-specifications, j
sonschema, nbformat, nbclient, nbconvert
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 17/17 [nbconvert]17 [jsonsc
hema-specifications]
Successfully installed beautifulsoup4-4.14.3 bleach-6.3.0 defusedxml-0.
7.1 fastjsonschema-2.21.2 jsonschema-4.26.0 jsonschema-specifications-2
025.9.1 jupyterlab-pygments-0.3.0 mistune-3.2.0 nbclient-0.10.4 nbconve
rt-7.17.0 nbformat-5.10.4 pandocfilters-1.5.1 referencing-0.37.0 rpds-p
y-0.30.0 soupsieve-2.8.3 tinycss2-1.4.0 webencodings-0.5.1
[NbConvertApp] Converting notebook cath20_analysis.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 28 image(s).
[NbConvertApp] Writing 2508750 bytes to cath20_analysis.html
```